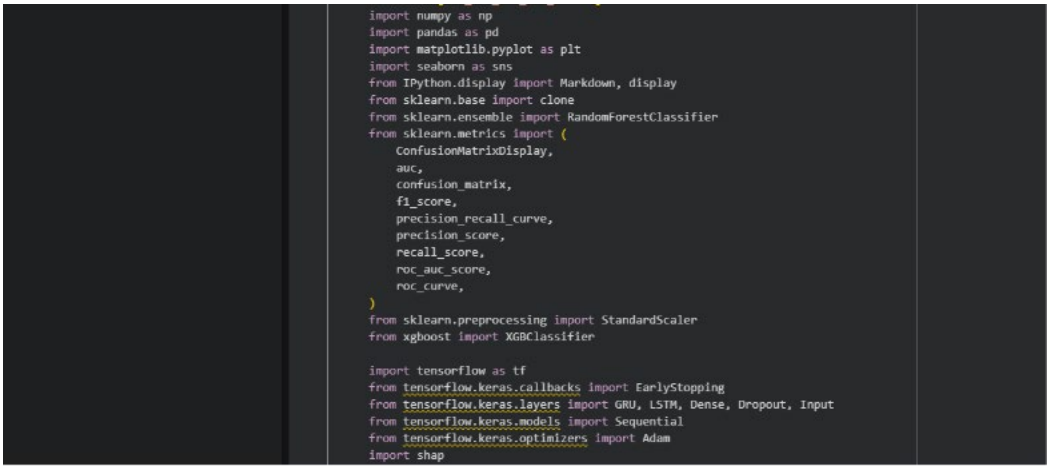
BAB IV

HASIL PENELITIAN DAN PEMBAHASAN

Bab ini menyajikan hasil penelitian berdasarkan tahapan eksperimen yang telah dirancang pada BAB III. Penyajian hasil dilakukan secara berurutan mulai dari lingkungan komputasi, pemuatan dataset, pemahaman struktur data, seleksi periode, pembersihan data, pembentukan target biner, penanganan nilai kosong, feature engineering, pembagian data, pelatihan model, evaluasi model, hingga interpretasi kontribusi fitur menggunakan SHAP. Urutan tersebut digunakan agar seluruh hasil penelitian dapat ditelusuri dari proses komputasi yang dijalankan pada notebook sampai pada output yang ditampilkan dalam bentuk tabel dan visualisasi.

4.1. Library dan Lingkungan Komputasi

Eksperimen penelitian ini dijalankan menggunakan bahasa pemrograman Python pada lingkungan notebook berbasis Google Colab. Lingkungan notebook dipilih karena mendukung eksekusi kode secara bertahap, dokumentasi proses eksperimen, serta penyimpanan output dalam bentuk tabel maupun gambar. Notebook eksperimen digunakan sebagai media utama untuk melakukan pemuatan data, pembersihan data, pembentukan fitur, pelatihan model, evaluasi, dan interpretasi SHAP.



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import Markdown, display
from sklearn.base import clone
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    auc,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier

import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers import GRU, LSTM, Dense, Dropout, Input
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
import shap
```

Gambar IV. 1 Cuplikan Library Python yang Digunakan dalam Eksperimen

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Gambar IV.1 memperlihatkan library Python yang digunakan untuk mendukung seluruh tahapan eksperimen. Library pandas digunakan untuk membaca dataset, mengelola struktur data berbentuk DataFrame, menghitung ringkasan data, dan menyimpan output tabel. Library numpy digunakan untuk operasi numerik, sedangkan matplotlib dan seaborn digunakan untuk membangun visualisasi seperti grafik distribusi kategori, heatmap missing values, confusion matrix, ROC curve, Precision-Recall curve, dan grafik SHAP feature importance.

Tahap pemodelan menggunakan beberapa library machine learning dan deep learning. Library scikit-learn digunakan untuk preprocessing, scaling, pemodelan Random Forest, serta penghitungan metrik evaluasi seperti Precision, Recall, F1-Score, ROC-AUC, dan confusion matrix. Library xgboost digunakan untuk membangun model XGBoost sebagai model ensemble pembanding. Sementara itu, tensorflow.keras digunakan untuk membangun model LSTM dan GRU karena kedua model tersebut memerlukan arsitektur neural network berbasis sequence. Library shap digunakan untuk menjelaskan kontribusi fitur terhadap keputusan model terpilih.

Tabel IV. 1 Library Utama yang Digunakan dalam Eksperimen

Library	Fungsi dalam Penelitian
pandas	Membaca dataset, mengolah DataFrame, menghitung ringkasan data, dan menyimpan tabel output.
numpy	Mendukung operasi numerik dan manipulasi array.
matplotlib	Membuat visualisasi grafik dasar.
seaborn	Membuat visualisasi statistik seperti heatmap, barplot, dan confusion matrix.
scikit-learn	Melakukan preprocessing, scaling, pemodelan Random Forest, dan evaluasi metrik klasifikasi.
xgboost	Membangun model XGBoost sebagai model ensemble pembanding.
tensorflow.keras	Membangun model LSTM dan GRU berbasis deep learning.
shap	Menginterpretasikan kontribusi fitur terhadap keputusan model terpilih.

Sumber: Diolah oleh penulis (2026)

Tabel IV.1 menunjukkan bahwa eksperimen tidak hanya berfokus pada proses pemodelan, tetapi juga mencakup pengolahan data, visualisasi, evaluasi, dan interpretasi model. Penggunaan beberapa library tersebut disesuaikan dengan kebutuhan penelitian. Library berbasis machine learning digunakan untuk membangun model klasifikasi, sedangkan SHAP digunakan untuk menjawab kebutuhan interpretasi kontribusi fitur yang menjadi bagian penting dalam penelitian ini. Notebook eksperimen dapat ditelusuri melalui tautan khusus berikut: [https://drive.google.com/file/d/1zoZmP6O0RCxMeGiFhYcaEb8zaPdGfwP4/view?usp=drive_link].

4.2. Load Dataset dan Gambaran Dataset Mentah

Dataset yang digunakan dalam penelitian ini adalah data Indeks Standar Pencemar Udara (ISPU) DKI Jakarta yang tersimpan dalam file ispu_dki_all.csv. Dataset tersebut merupakan data sekunder yang memuat observasi harian kualitas udara berdasarkan beberapa parameter polutan. Sumber data asli dicantumkan secara langsung agar dataset yang digunakan dapat ditelusuri. Dataset terstruktur diperoleh dari Kaggle melalui tautan

https://www.kaggle.com/datasets/senadu34/air-quality-index-in-jakarta-2010-2021/data. Sumber pemerintah yang relevan berasal dari Portal Satu Data Jakarta melalui tautan https://satudata.jakarta.go.id/search?q=ispu&halaman=all&kategori=dataset&topik=all&organisasi=all&status=all&sort=desc&page_no=1&show_filter=true&lang=id. Secara substantif, data ISPU tersebut berkaitan dengan data kualitas udara yang dikelola oleh Dinas Lingkungan Hidup Provinsi DKI Jakarta. File dataset mentah yang digunakan dalam eksperimen dapat diakses melalui tautan berikut: [https://drive.google.com/file/d/10GA-CjmIl_6htHX4n-c6XeveHduLhQ9J/view?usp=drive_link].



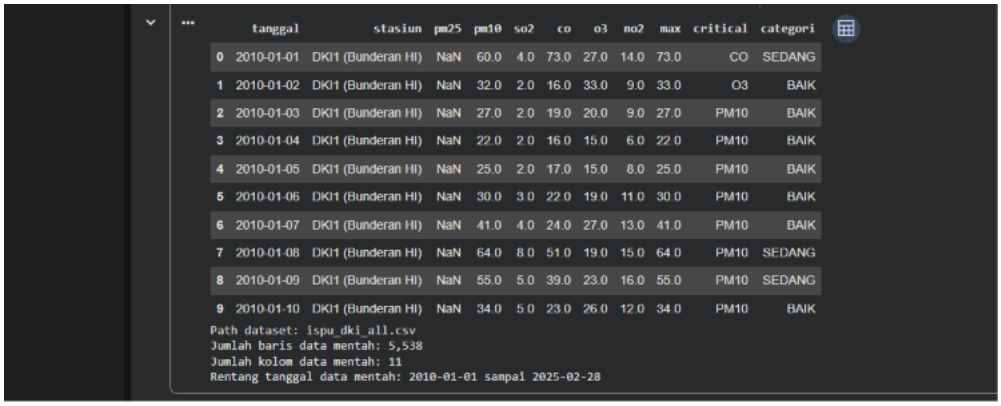
```
raw_df = pd.read_csv(dataset_path)
raw_df["tanggal"] = pd.to_datetime(raw_df["tanggal"], errors="coerce")

display(raw_df.head(10))
print(f"Path dataset: {dataset_path}")
print(f"Jumlah baris data mentah: {raw_df.shape[0]:,}")
print(f"Jumlah kolom data mentah: {raw_df.shape[1]:,}")
print(f"Rentang tanggal data mentah: {raw_df['tanggal'].min().date()} sampai {raw_df['tanggal'].max().date()}")
```

Gambar IV. 2 Cuplikan Kode Python untuk Memuat Dataset Mentah

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Gambar IV.2 memperlihatkan proses awal pemuatan dataset ke dalam notebook. File ispu_dki_all.csv dibaca menggunakan fungsi read_csv() dari library pandas, kemudian kolom tanggal dikonversi ke format waktu menggunakan pd.to_datetime(). Konversi ini penting karena kolom tanggal tidak hanya berfungsi sebagai informasi administratif, tetapi juga digunakan untuk seleksi periode penelitian, pengurutan data secara kronologis, pembentukan fitur waktu, dan pembagian data latih serta data uji. Setelah data berhasil dimuat, notebook menampilkan beberapa baris awal dataset menggunakan head() dan memeriksa jumlah baris, jumlah kolom, serta rentang tanggal dataset.



	tanggal	stasiun	pm25	pm10	so2	co	o3	no2	max	critical	categori
0	2010-01-01	DKI1 (Bunderan HI)	NaN	60.0	4.0	73.0	27.0	14.0	73.0	CO	SEDANG
1	2010-01-02	DKI1 (Bunderan HI)	NaN	32.0	2.0	16.0	33.0	9.0	33.0	O3	BAIK
2	2010-01-03	DKI1 (Bunderan HI)	NaN	27.0	2.0	19.0	20.0	9.0	27.0	PM10	BAIK
3	2010-01-04	DKI1 (Bunderan HI)	NaN	22.0	2.0	16.0	15.0	6.0	22.0	PM10	BAIK
4	2010-01-05	DKI1 (Bunderan HI)	NaN	25.0	2.0	17.0	15.0	8.0	25.0	PM10	BAIK
5	2010-01-06	DKI1 (Bunderan HI)	NaN	30.0	3.0	22.0	19.0	11.0	30.0	PM10	BAIK
6	2010-01-07	DKI1 (Bunderan HI)	NaN	41.0	4.0	24.0	27.0	13.0	41.0	PM10	BAIK
7	2010-01-08	DKI1 (Bunderan HI)	NaN	64.0	8.0	51.0	19.0	15.0	64.0	PM10	SEDANG
8	2010-01-09	DKI1 (Bunderan HI)	NaN	55.0	5.0	39.0	23.0	16.0	55.0	PM10	SEDANG
9	2010-01-10	DKI1 (Bunderan HI)	NaN	34.0	5.0	23.0	26.0	12.0	34.0	PM10	BAIK

```
Path dataset: ispu_dki_all.csv
Jumlah baris data mentah: 5,538
Jumlah kolom data mentah: 11
Rentang tanggal data mentah: 2010-01-01 sampai 2025-02-28
```

Gambar IV. 3 Screenshot Dataset Mentah ispu_dki_all.csv
Sumber: Diolah oleh penulis (2026)

Output pada Gambar IV.3 menunjukkan bahwa dataset mentah terdiri atas kolom tanggal, stasiun pemantauan, enam parameter polutan, nilai maksimum, parameter kritis, dan kategori ISPU asli. Enam parameter polutan yang tersedia adalah PM2.5, PM10, SO2, CO, O3, dan NO2. Kolom-kolom polutan tersebut menjadi kandidat fitur utama dalam pemodelan karena merepresentasikan kondisi kualitas udara. Kolom categori digunakan sebagai sumber pembentukan target klasifikasi, sedangkan kolom max dan critical tidak digunakan sebagai fitur model karena keduanya berhubungan langsung dengan pembentukan kategori ISPU.

Tabel IV. 2 Ringkasan Dataset Mentah

Komponen	Nilai
Jumlah baris	5.538
Jumlah kolom	11
Tanggal awal	2010-01-01
Tanggal akhir	2025-02-28
Kolom tersedia	tanggal, stasiun, pm25, pm10, so2, co, o3, no2, max, critical, categori

Sumber: Diolah oleh penulis (2026)

Tabel IV.2 menunjukkan bahwa dataset mentah terdiri dari 5.538 baris dan 11 kolom dengan rentang tanggal mulai 1 Januari 2010 sampai 28 Februari 2025. Jumlah observasi tersebut diperoleh dari pemeriksaan dimensi data menggunakan atribut shape pada DataFrame. Nilai 5.538 menunjukkan jumlah baris observasi harian yang tersedia sebelum seleksi periode penelitian dilakukan, sedangkan nilai 11 menunjukkan jumlah kolom yang tersedia pada dataset

mentah. Karena periode penelitian dibatasi pada tahun 2020 sampai 2024, tidak seluruh baris pada dataset mentah digunakan dalam pemodelan.

4.3. Data Understanding dan Struktur Dataset

Tahap data understanding dilakukan untuk memastikan bahwa dataset memiliki struktur kolom yang sesuai dengan kebutuhan penelitian. Pemeriksaan ini mencakup kelengkapan nama kolom, tipe data, jumlah nilai non-null, dan peran setiap kolom dalam alur penelitian. Tahap ini penting karena kesalahan dalam memahami peran kolom dapat menyebabkan fitur yang tidak semestinya masuk ke dalam model, terutama kolom yang berpotensi menyebabkan target leakage.



```
[5]
✓ 0s
    expected_columns = {"tanggal", "stasiun", "pm25", "pm10", "so2", "co", "o3", "no2", "max", "critical", "categori"}
    missing_columns = expected_columns.difference(raw_df.columns)
    if missing_columns:
        raise ValueError(f"Kolom dataset belum sesuai: {sorted(missing_columns)}")

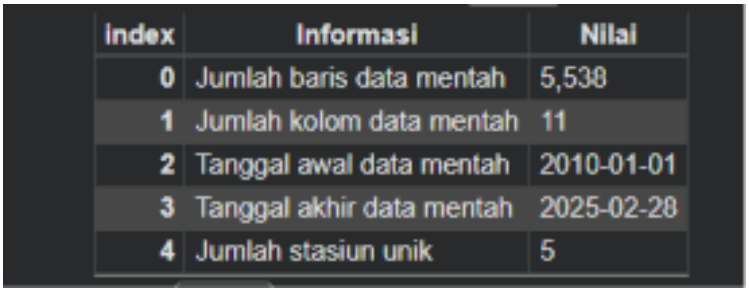
    data_structure = pd.DataFrame({
        "Informasi": [
            "Jumlah baris data mentah",
            "Jumlah kolom data mentah",
            "Tanggal awal data mentah",
            "Tanggal akhir data mentah",
            "Jumlah stasiun unik",
        ],
        "Nilai": [
            f"{raw_df.shape[0]:,}",
            f"{raw_df.shape[1]:,}",
            raw_df["tanggal"].min().date(),
            raw_df["tanggal"].max().date(),
            raw_df["stasiun"].nunique(dropna=True),
        ],
    })
    save_table(data_structure, "01_ringkasan_struktur_data.csv")
    display(data_structure)

    df_period_raw = (
        raw_df.loc[(raw_df["tanggal"] >= START_DATE) & (raw_df["tanggal"] <= END_DATE)]
        .copy()
        .sort_values("tanggal")
        .reset_index(drop=True)
    )
```

Gambar IV. 4 Cuplikan Kode Python Pemeriksaan Struktur Dataset

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.4 digunakan untuk memeriksa apakah seluruh kolom yang dibutuhkan tersedia di dalam dataset. Pemeriksaan dilakukan dengan membandingkan daftar kolom yang diharapkan dengan kolom aktual pada DataFrame. Jika terdapat kolom yang tidak tersedia, proses analisis perlu dihentikan atau diperbaiki karena dapat memengaruhi tahapan berikutnya. Selain itu, notebook juga memeriksa jumlah nilai non-null pada setiap kolom untuk melihat kelengkapan data awal sebelum analisis missing values dilakukan secara lebih rinci.



index	Informasi	Nilai
0	Jumlah baris data mentah	5,538
1	Jumlah kolom data mentah	11
2	Tanggal awal data mentah	2010-01-01
3	Tanggal akhir data mentah	2025-02-28
4	Jumlah stasiun unik	5

Gambar IV. 5 Screenshot Output Pemeriksaan Struktur Dataset

Sumber: Diolah oleh penulis (2026)

Output pada Gambar IV.5 menunjukkan bahwa dataset memiliki 11 kolom utama. Kolom tanggal berperan sebagai dasar pengurutan waktu, seleksi periode, pembentukan fitur temporal, dan split kronologis. Kolom stasiun memuat informasi lokasi pemantauan, tetapi tidak digunakan sebagai fitur model karena penelitian ini tidak diarahkan untuk membandingkan kualitas udara antarstasiun. Enam kolom polutan digunakan sebagai fitur utama karena mewakili parameter kualitas udara. Kolom categori digunakan sebagai sumber pembentukan target biner, sedangkan max dan critical dikeluarkan dari fitur model untuk menghindari target leakage.

Tabel IV. 3 Peran Kolom Dataset dalam Penelitian

Kolom	Peran dalam Penelitian	Keputusan Penggunaan
tanggal	Dasar seleksi periode, pengurutan kronologis, pembentukan fitur waktu, dan split data.	Digunakan secara tidak langsung.
stasiun	Informasi lokasi stasiun pemantauan.	Tidak digunakan sebagai fitur karena penelitian tidak membandingkan antarstasiun.
pm25	Parameter polutan PM2.5.	Digunakan sebagai fitur model.
pm10	Parameter polutan PM10.	Digunakan sebagai fitur model.
so2	Parameter polutan sulfur dioksida.	Digunakan sebagai fitur model.
co	Parameter polutan karbon monoksida.	Digunakan sebagai fitur model.
o3	Parameter polutan ozon.	Digunakan sebagai fitur model.

Kolom	Peran dalam Penelitian	Keputusan Penggunaan
no2	Parameter polutan nitrogen dioksida.	Digunakan sebagai fitur model.
max	Nilai maksimum parameter pembentuk kategori ISPU.	Tidak digunakan karena berpotensi menyebabkan target leakage.
critical	Parameter kritis yang berkaitan dengan pembentukan kategori ISPU.	Tidak digunakan karena berpotensi menyebabkan target leakage.
categori	Kategori ISPU asli.	Digunakan sebagai sumber target, tetapi tidak digunakan sebagai fitur model.

Sumber: Diolah oleh penulis (2026)

Tabel IV.3 menegaskan bahwa tidak semua kolom pada dataset mentah digunakan sebagai input model. Kolom polutan digunakan sebagai fitur utama, sedangkan kolom target dan kolom yang terlalu dekat dengan proses pembentukan target dikeluarkan dari daftar fitur. Keputusan untuk tidak menggunakan max dan critical dilakukan untuk menjaga validitas pemodelan. Apabila kedua kolom tersebut dimasukkan sebagai fitur, model berpotensi mempelajari informasi yang sudah sangat dekat dengan label kategori ISPU, sehingga performa evaluasi dapat terlihat tinggi tetapi tidak merepresentasikan kemampuan generalisasi yang wajar. File pendukung untuk struktur dataset awal dapat disediakan melalui tautan berikut: [https://drive.google.com/file/d/1xFKfjtj0mlLO_lId6zAe3rrAV1EsjALO/view?usp=sharing].

4.4. Seleksi Periode, Pembersihan, dan Data Final 1.826 Observasi

Seleksi periode dilakukan untuk menyesuaikan dataset dengan batasan penelitian, yaitu 1 Januari 2020 sampai 31 Desember 2024. Dataset mentah awalnya memuat 5.538 baris dengan rentang tahun 2010 sampai 2025. Karena penelitian ini hanya menganalisis periode 2020-2024, notebook memfilter kolom tanggal agar hanya observasi dalam rentang tersebut yang digunakan. Setelah proses seleksi periode, ditemukan satu baris dengan kategori TIDAK ADA DATA. Baris tersebut dikeluarkan karena tidak dapat dipetakan ke dalam kelas target biner yang valid.



```
[6]
✓ 0s
df_period = df_period_raw.copy()
df_period = df_period.sort_values("tanggal").reset_index(drop=True)

analysis_df = df_period.loc[df_period["categori"] != "TIDAK ADA DATA"].copy()
analysis_df = analysis_df.reset_index(drop=True)

period_summary = pd.DataFrame({
    "Informasi": [
        "Tanggal awal penelitian",
        "Tanggal akhir penelitian",
        "Jumlah observasi periode 2020-2024",
        "Jumlah observasi valid untuk pemodelan",
        "Kategori yang dikeluarkan",
    ],
    "Nilai": [
        df_period["tanggal"].min().date(),
        df_period["tanggal"].max().date(),
        f"{len(df_period):,}",
        f"{len(analysis_df):,}",
        "TIDAK ADA DATA",
    ],
})
save_table(period_summary, "03_ringkasan_seleksi_periode.csv")
display(period_summary)

assert df_period["tanggal"].min() == pd.Timestamp(START_DATE)
```

Gambar IV. 6 Cuplikan Kode Python Seleksi Periode dan Pembersihan Data

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.6 bekerja dengan cara memilih baris yang nilai tanggalnya berada di antara 2020-01-01 dan 2024-12-31. Data hasil filter disalin ke DataFrame baru agar proses pembersihan tidak mengubah dataset mentah. Selanjutnya, data dengan kategori TIDAK ADA DATA dikeluarkan menggunakan kondisi pada kolom categori. Tahap ini menghasilkan data final yang digunakan untuk pembentukan target biner, pemeriksaan nilai kosong, feature engineering, split data, pemodelan, dan evaluasi.

Hasil Seleksi Periode dan Pembersihan Data

Tahap	Jumlah Observasi	Rentang Tanggal	Keterangan
Dataset awal	5538	1 Januari 2010 - 28 Februari 2025	Data mentah
Setelah seleksi periode 2020-2024	1827	1 Januari 2020 - 31 Desember 2024	Data sesuai periode penelitian
Setelah mengeluarkan TIDAK ADA DATA	1826	1 Januari 2020 - 31 Desember 2024	Data final pemodelan

Gambar IV. 7 Screenshot Output Seleksi Periode dan Data Final Pemodelan

Sumber: Diolah oleh penulis (2026)

Gambar IV.7 menunjukkan perubahan jumlah observasi setelah seleksi periode dan pembersihan data. Dataset awal berjumlah 5.538 baris. Setelah dibatasi pada periode 2020-2024, jumlah data menjadi 1.827 observasi. Satu baris kategori TIDAK ADA DATA kemudian dikeluarkan sehingga data final pemodelan berjumlah 1.826 observasi. Dengan demikian,

seluruh tahapan pemodelan dan evaluasi setelah tahap ini menggunakan 1.826 observasi sebagai data valid.



	Informasi	Nilai
0	Tanggal awal penelitian	2020-01-01
1	Tanggal akhir penelitian	2024-12-31
2	Jumlah observasi periode 2020-2024	1,827
3	Jumlah observasi valid untuk pemodelan	1,826
4	Kategori yang dikeluarkan	TIDAK ADA DATA

Gambar IV. 8 Screenshot Shape Data Final Pemodelan 1.826 Observasi

Sumber: Diolah oleh penulis dari output notebook (2026)

Output pada Gambar IV.8 memperkuat hasil seleksi periode dengan menampilkan bentuk akhir DataFrame yang digunakan untuk pemodelan. Informasi shape data final menunjukkan bahwa jumlah observasi yang dipakai dalam eksperimen adalah 1.826 baris. Data final tersebut dapat ditelusuri pada file khusus berikut: [https://drive.google.com/file/d/1KwAZ1qfPZPVIafdAB4Ly_rRqr9a9KXtT/view?usp=drive_link]. Sementara itu, data hasil filter periode sebelum pengeluaran kategori TIDAK ADA DATA dapat dicantumkan melalui tautan berikut: [https://drive.google.com/file/d/1rXm_5iphoGwPe9o_TQtDN1RS9zz0Rt9H/view?usp=drive_link].

Tabel IV. 4 Ringkasan Seleksi Periode dan Pembersihan Data

Tahap	Jumlah Observasi	Keterangan
Dataset awal	5.538	Data harian ISPU pada rentang 2010-01-01 sampai 2025-02-28.
Filter periode 2020-2024	1.827	Data yang berada pada rentang 2020-01-01 sampai 2024-12-31.
Pengeluaran kategori TIDAK ADA DATA	1.826	Satu baris tidak valid dikeluarkan karena tidak dapat digunakan sebagai target.

Tahap	Jumlah Observasi	Keterangan
Data final pemodelan	1.826	Data valid untuk target biner, pemodelan, evaluasi, dan SHAP.

Sumber: Diolah oleh penulis (2026)

Tabel IV.4 memperlihatkan bahwa proporsi data periode penelitian terhadap dataset mentah adalah 1.827 dibagi 5.538 kemudian dikalikan 100%, sehingga menghasilkan 32,99%. Setelah satu observasi kategori TIDAK ADA DATA dikeluarkan, jumlah data final diperoleh dari 1.827 dikurangi 1, yaitu 1.826 observasi. Angka 1.826 tersebut menjadi penyebut utama dalam perhitungan distribusi target, persentase missing values, dan ringkasan data final pada tahapan berikutnya.

4.5. Distribusi Kategori ISPU Asli

Distribusi kategori ISPU asli diperlukan untuk memahami komposisi kelas sebelum kategori tersebut dipetakan menjadi target biner. Pada notebook, distribusi awal dihitung dari data periode 2020-2024, kemudian hasil final yang digunakan untuk pemodelan dihitung setelah baris TIDAK ADA DATA dikeluarkan. Dengan cara ini, pembahasan tetap menunjukkan kondisi kategori asli, tetapi perhitungan proporsi yang menjadi dasar pemodelan menggunakan 1.826 observasi valid.

```
[7]
✓ 0s
analysis_df["target_tidak_sehat"] = analysis_df["categori"].isin(target_positive_categories).astype(int)
analysis_df["target_label"] = analysis_df["target_tidak_sehat"].map({
    0: "Bukan Tidak Sehat+",
    1: "Tidak Sehat+",
})

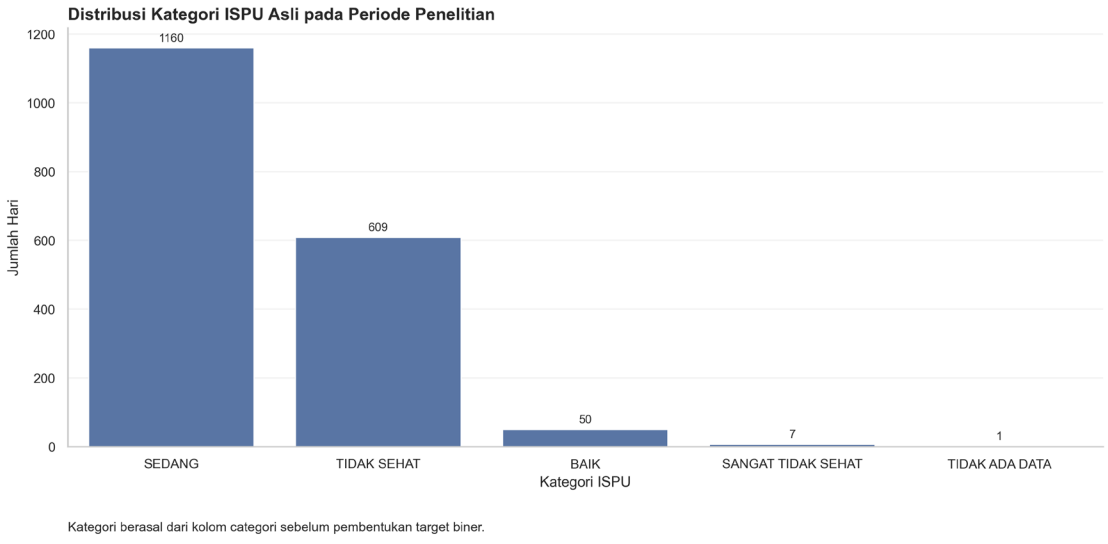
target_counts = (
    analysis_df["target_label"]
    .value_counts()
    .rename_axis("Target Klasifikasi")
    .reset_index(name="Jumlah Hari")
)
target_counts["Persentase"] = (target_counts["Jumlah Hari"] / target_counts["Jumlah Hari"].sum() * 100).round(2)
save_table(target_counts, "04_distribusi_target_biner.csv")
display(target_counts)

fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(data=target_counts, x="Target Klasifikasi", y="Jumlah Hari", ax=ax)
ax.set_title("Distribusi Target Biner ISPU", loc="left", fontweight="bold")
clean_axes(ax, xlabel="Target", ylabel="Jumlah Hari")
add_bar_labels(ax)
add_note(ax, "Tidak Sehat+ adalah kelas positif; Bukan Tidak Sehat+ adalah kelas pembanding.")
plt.tight_layout()
save_figure(fig, "02_distribusi_target_biner.png")
plt.show()
```

Gambar IV. 9 Cuplikan Kode Python Distribusi Kategori ISPU Asli

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.9 menghitung frekuensi kategori ISPU menggunakan fungsi value_counts() pada kolom categori. Frekuensi tersebut kemudian diubah menjadi persentase dengan membagi jumlah hari pada setiap kategori terhadap total observasi yang dianalisis. Pada data final pemodelan, kategori TIDAK ADA DATA sudah tidak dimasukkan sehingga proporsi kategori valid dihitung terhadap 1.826 observasi.



Gambar IV. 10 Screenshot Output Distribusi Kategori ISPU Asli

Sumber: Diolah oleh penulis (2026)

Gambar IV.10 masih memperlihatkan distribusi kategori awal pada periode 2020-2024 yang mencakup satu baris TIDAK ADA DATA. Informasi tersebut berguna untuk menunjukkan bahwa baris tidak valid memang ada pada data hasil seleksi periode. Namun, karena tahap pemodelan hanya menggunakan kategori yang valid, perhitungan akhir pada Tabel IV.5 menggunakan 1.826 observasi setelah TIDAK ADA DATA dikeluarkan.

Tabel IV. 5 Distribusi Kategori ISPU Valid pada Data Final 1.826 Observasi

Kategori ISPU	Jumlah Hari	Perhitungan Persentase	Persentase
SEDANG	1.160	1.160 / 1.826 × 100%	63,53%
TIDAK SEHAT	609	609 / 1.826 × 100%	33,35%
BAIK	50	50 / 1.826 × 100%	2,74%
SANGAT TIDAK SEHAT	7	7 / 1.826 × 100%	0,38%

Sumber: Diolah oleh penulis (2026)

Tabel IV.5 menunjukkan bahwa kategori SEDANG merupakan kategori paling dominan pada data final, yaitu 1.160 hari. Persentase kategori SEDANG dihitung dengan membagi 1.160 terhadap total 1.826 observasi, kemudian dikalikan 100%, sehingga menghasilkan 63,53%. Kategori TIDAK SEHAT berjumlah 609 hari; perhitungannya adalah 609 dibagi 1.826 kemudian dikalikan 100%, sehingga menghasilkan 33,35%. Kategori BAIK berjumlah 50 hari; perhitungannya adalah 50 dibagi 1.826 kemudian dikalikan 100%, sehingga menghasilkan 2,74%. Kategori SANGAT TIDAK SEHAT berjumlah 7 hari; perhitungannya adalah 7 dibagi 1.826 kemudian dikalikan 100%, sehingga menghasilkan 0,38%. Hasil ini menunjukkan bahwa kategori SEDANG mendominasi data final, sedangkan kategori TIDAK SEHAT tidak lagi dihitung dalam distribusi ini.

4.6. Pembentukan Target Biner

Pembentukan target biner dilakukan karena penelitian ini berfokus pada klasifikasi kondisi kualitas udara yang perlu mendapat perhatian. Kategori Tidak Sehat+ dan Bukan Tidak Sehat+. Kelas Tidak Sehat+ dibentuk dari kategori asli TIDAK SEHAT dan SANGAT TIDAK

SEHAT, sedangkan kelas Bukan Tidak Sehat+ dibentuk dari kategori asli BAIK dan SEDANG. Penamaan kelas Bukan Tidak Sehat+ digunakan sebagai kelas pembanding agar definisi target tetap konsisten dengan struktur kategori pada kolom categori. Dalam konteks ini, SANGAT TIDAK SEHAT tetap dipahami sebagai kategori asli ISPU, sedangkan Tidak Sehat+ merupakan definisi operasional penelitian dan bukan kategori resmi baru dalam regulasi ISPU. BAIK dan SEDANG dipetakan menjadi kelas Bukan Tidak Sehat+, sedangkan kategori TIDAK SEHAT dan SANGAT TIDAK SEHAT dipetakan menjadi kelas Tidak Sehat+. Istilah Tidak Sehat+ merupakan definisi operasional penelitian untuk menggabungkan kategori yang menunjukkan kondisi udara tidak sehat atau lebih buruk, bukan kategori resmi baru dalam regulasi ISPU.



```
[7]
✓ 0s
    analysis_df["target_tidak_sehat"] = analysis_df["categori"].isin(target_positive_categories).astype(int)
    analysis_df["target_label"] = analysis_df["target_tidak_sehat"].map({
        0: "Bukan Tidak Sehat+",
        1: "Tidak Sehat+",
    })

    target_counts = (
        analysis_df["target_label"]
        .value_counts()
        .rename_axis("Target Klasifikasi")
        .reset_index(name="Jumlah Hari")
    )
    target_counts["Persentase"] = (target_counts["Jumlah Hari"] / target_counts["Jumlah Hari"].sum() * 100).round(2)
    save_table(target_counts, "04_distribusi_target_biner.csv")
    display(target_counts)

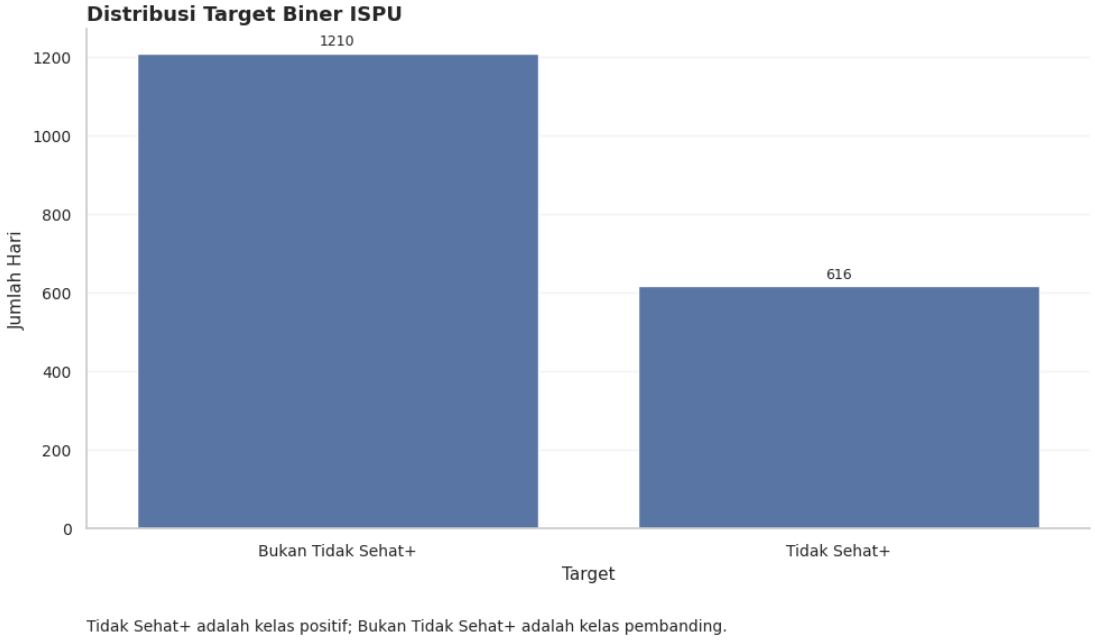
    fig, ax = plt.subplots(figsize=(10, 6))
    sns.barplot(data=target_counts, x="Target Klasifikasi", y="Jumlah Hari", ax=ax)
    ax.set_title("Distribusi Target Biner ISPU", loc="left", fontweight="bold")
    clean_axes(ax, xlabel="Target", ylabel="Jumlah Hari")
    add_bar_labels(ax)
    add_note(ax, "Tidak Sehat+ adalah kelas positif; Bukan Tidak Sehat+ adalah kelas pembanding.")
    plt.tight_layout()
    save_figure(fig, "02_distribusi_target_biner.png")
    plt.show()
```

Gambar IV. 11 Cuplikan Kode Python Pembentukan Target Biner

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.11 membentuk aturan pemetaan dari kategori asli ISPU ke dua kelas target biner. Kategori BAIK dan SEDANG diberi nilai 0 sebagai kelas Bukan Tidak Sehat+, sedangkan kategori TIDAK SEHAT dan SANGAT TIDAK SEHAT diberi nilai 1 sebagai kelas Tidak Sehat+. Kolom target_tidak_sehat digunakan sebagai target numerik dalam pelatihan model, sedangkan target_label digunakan untuk menampilkan nama kelas target pada tabel dan grafik. Karena kategori TIDAK ADA DATA telah dikeluarkan pada tahap

pembersihan data, kategori tersebut tidak masuk ke dalam DataFrame pemodelan dan tidak menjadi bagian dari distribusi target biner.



Gambar IV. 12 Screenshot Output Pemetaan Target Biner

Sumber: Diolah oleh penulis (2026)

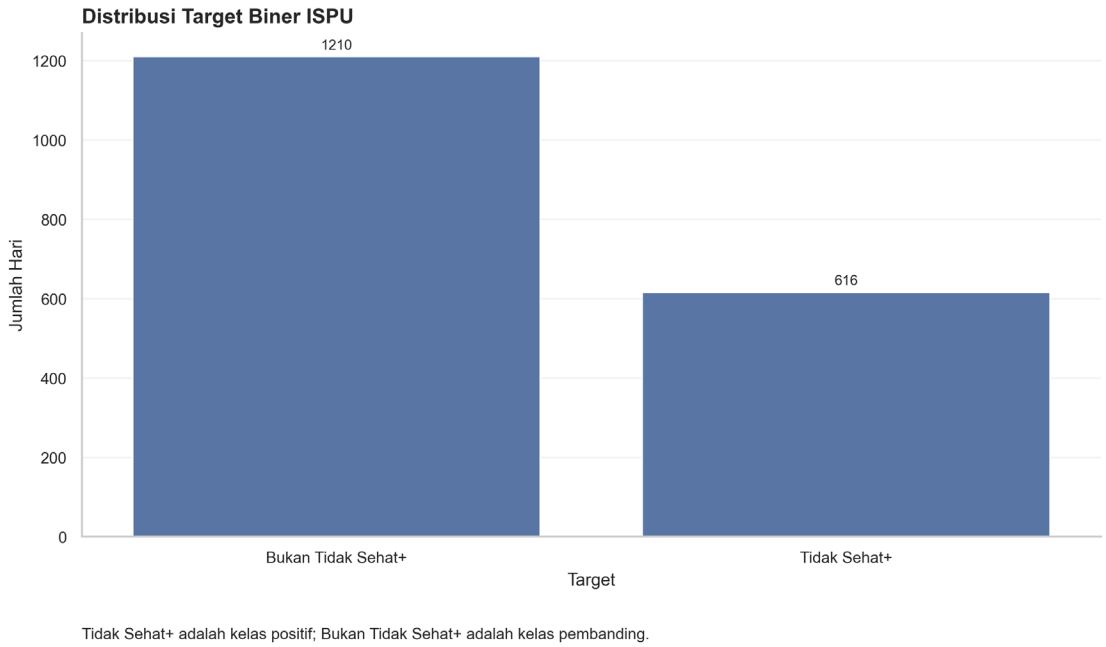
Gambar IV.12 memperlihatkan aturan pemetaan dari kategori ISPU asli ke target biner penelitian. Pemetaan ini menjaga agar target model tetap bermakna secara substantif. Kelas positif berisi kategori yang mengindikasikan kualitas udara tidak sehat atau lebih buruk, sedangkan kelas negatif berisi kategori yang belum masuk ke kondisi tidak sehat.

Tabel IV. 6 Pemetaan Target Biner

Kategori Asli	**Target Biner**	**Nilai Target**	Keterangan
BAIK	Bukan Tidak Sehat+	0	Kelas pembanding/negatif
SEDANG	Bukan Tidak Sehat+	0	Kelas pembanding/negatif
TIDAK SEHAT	Tidak Sehat+	1	Kelas positif
SANGAT TIDAK SEHAT	Tidak Sehat+	1	Kelas positif; tetap kategori asli ISPU
TIDAK ADA DATA	Dikeluarkan	-	Tidak digunakan dalam pemodelan

Sumber: Diolah oleh penulis (2026)

Tabel IV.6 tidak menambah kategori ISPU baru, tetapi menjelaskan bagaimana empat kategori asli yang valid dikelompokkan menjadi dua kelas target biner. Kategori BAIK dan SEDANG menjadi kelas Bukan Tidak Sehat+ karena keduanya berperan sebagai kelas pembanding, sedangkan TIDAK SEHAT dan SANGAT TIDAK SEHAT menjadi kelas Tidak Sehat+ karena keduanya mewakili kondisi kualitas udara yang menjadi perhatian utama model. File mapping target biner dapat dilampirkan melalui tautan berikut: [https://drive.google.com/file/d/1jm-xBNlS_cq_xAgYImqOpsAGbynojXUe/view?usp=drive_link].



Gambar IV. 13 Screenshot Output Distribusi Target Biner

Sumber: Diolah oleh penulis (2026)

Gambar IV.13 menampilkan distribusi target setelah pemetaan biner dilakukan. Kelas Bukan Tidak Sehat+ terlihat lebih besar daripada kelas Tidak Sehat+, sehingga data target tidak sepenuhnya seimbang. Kondisi ini menjadi dasar mengapa evaluasi model tidak hanya dibaca dari satu metrik, tetapi menggunakan Precision, Recall, F1-Score, dan ROC-AUC secara bersamaan.

Tabel IV. 7 Distribusi Target Biner

Target Biner	Jumlah Hari	Perhitungan Persentase	Persentase
Bukan Tidak Sehat+	1.210	1.210 / 1.826 × 100%	66,27%
Tidak Sehat+	616	616 / 1.826 × 100%	33,73%

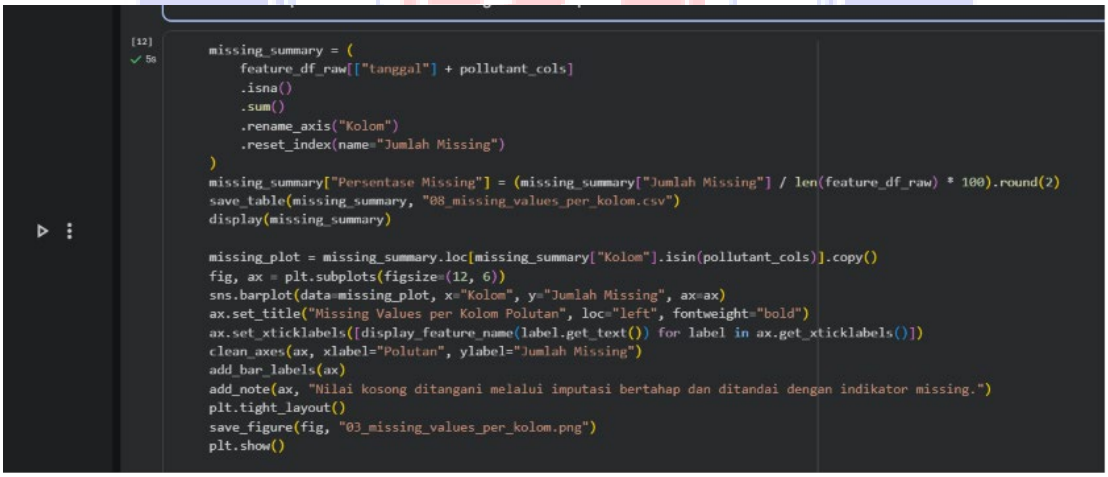
Sumber: Diolah oleh penulis (2026)

Tabel IV.7 menunjukkan bahwa kelas Bukan Tidak Sehat+ berjumlah 1.210 observasi. Persentasenya dihitung dengan membagi 1.210 terhadap total 1.826 observasi, kemudian dikalikan 100%, sehingga menghasilkan 66,27%. Kelas Tidak Sehat+ berjumlah 616 observasi. Persentasenya dihitung dengan membagi 616 terhadap total 1.826 observasi, kemudian dikalikan 100%, sehingga menghasilkan 33,73%. Komposisi ini menunjukkan adanya

ketidakseimbangan kelas karena kelas Bukan Tidak Sehat+ lebih dominan daripada kelas Tidak Sehat+. Oleh karena itu, evaluasi model tidak hanya bertumpu pada satu metrik, tetapi membaca Precision, Recall, F1-Score, dan ROC-AUC secara bersamaan.Data target biner dapat ditelusuri melalui tautan berikut: [https://drive.google.com/file/d/17hhsfT7o6OTmn7UIAvbAXrDmdc9_s5rC/view?usp=drive_link].

4.7. Pemeriksaan Missing Values Awal

Pemeriksaan missing values dilakukan pada data final 1.826 observasi untuk mengetahui kelengkapan setiap parameter polutan. Pemeriksaan ini penting karena model machine learning membutuhkan input numerik yang lengkap. Selain itu, pola nilai kosong perlu dipahami agar proses imputasi tidak dilakukan secara sembarangan dan tetap mempertimbangkan karakter data deret waktu harian.



```
[12]
✓ 5s
    missing_summary = (
        feature_df_raw[["tanggal"] + pollutant_cols]
        .isna()
        .sum()
        .rename_axis("Kolom")
        .reset_index(name="Jumlah Missing")
    )
    missing_summary["Persentase Missing"] = (missing_summary["Jumlah Missing"] / len(feature_df_raw) * 100).round(2)
    save_table(missing_summary, "08_missing_values_per_kolom.csv")
    display(missing_summary)

    missing_plot = missing_summary.loc[missing_summary["Kolom"].isin(pollutant_cols)].copy()
    fig, ax = plt.subplots(figsize=(12, 6))
    sns.barplot(data=missing_plot, x="Kolom", y="Jumlah Missing", ax=ax)
    ax.set_title("Missing Values per Kolom Polutan", loc="left", fontweight="bold")
    ax.set_xticklabels([display_feature_name(label.get_text()) for label in ax.get_xticklabels()])
    clean_axes(ax, xlabel="Polutan", ylabel="Jumlah Missing")
    add_bar_labels(ax)
    add_note(ax, "Nilai kosong ditangani melalui imputasi bertahap dan ditandai dengan indikator missing.")
    plt.tight_layout()
    save_figure(fig, "03_missing_values_per_kolom.png")
    plt.show()
```

Gambar IV. 14 Cuplikan Kode Python Perhitungan Missing Values

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.14 menghitung jumlah nilai kosong pada setiap kolom polutan menggunakan fungsi isna().sum(). Setelah jumlah missing values diperoleh, persentasenya dihitung dengan membagi jumlah missing pada setiap kolom terhadap total data final 1.826

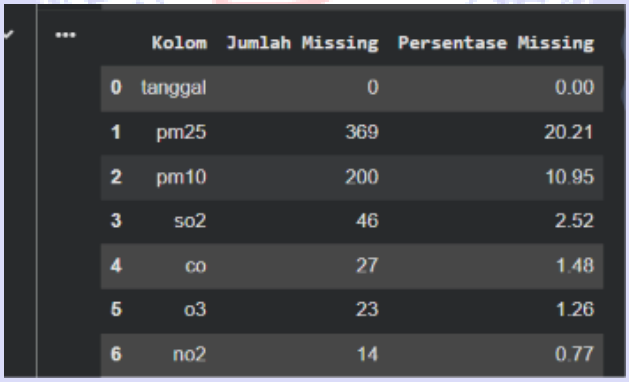
observasi. Perhitungan ini menghasilkan ukuran yang sebanding antarpolutan sehingga dapat diketahui polutan mana yang memiliki masalah kelengkapan data paling besar.

Tabel IV. 8 Missing Values pada Parameter Polutan

Polutan	Jumlah Missing	Perhitungan Persentase	Persentase Missing
PM2.5	369	(369 / 1.826) × 100%	20,21%
PM10	200	(200 / 1.826) × 100%	10,95%
SO2	46	(46 / 1.826) × 100%	2,52%
CO	27	(27 / 1.826) × 100%	1,48%
O3	23	(23 / 1.826) × 100%	1,26%
NO2	14	(14 / 1.826) × 100%	0,77%

Sumber: Diolah oleh penulis (2026)

Tabel IV.8 menunjukkan bahwa PM2.5 memiliki missing values terbesar, yaitu 369 nilai, PM10 memiliki 200 missing values, SO2 memiliki 46 missing values, CO memiliki 27 missing values, O3 memiliki 23 missing values, NO2 memiliki 14 missing values.

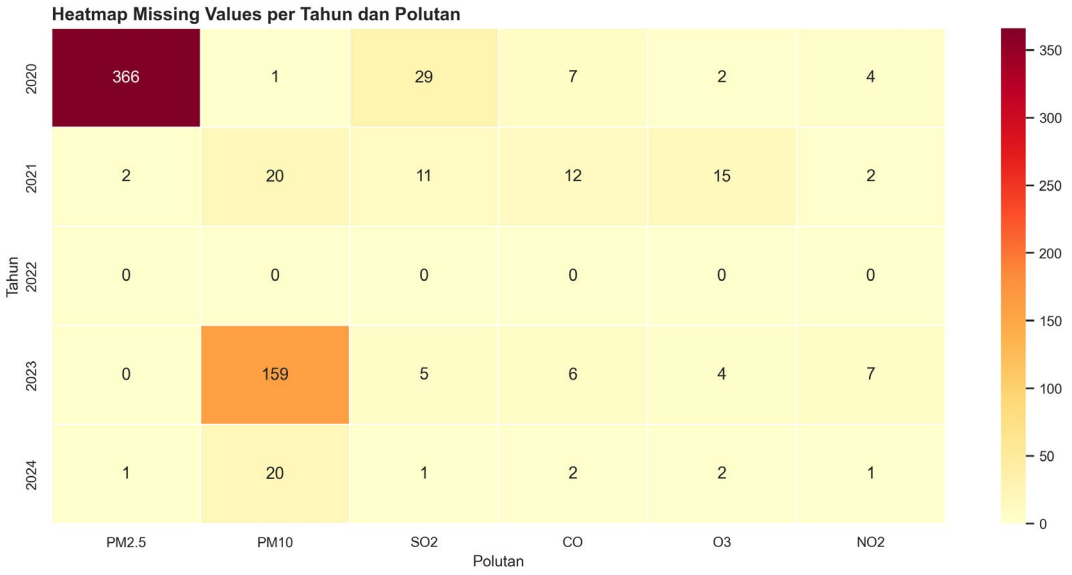


	Kolom	Jumlah Missing	Persentase Missing
0	tanggal	0	0.00
1	pm25	369	20.21
2	pm10	200	10.95
3	so2	46	2.52
4	co	27	1.48
5	o3	23	1.26
6	no2	14	0.77

Gambar IV. 15 Screenshot Output Tabel Missing Values per Polutan

Sumber: Diolah oleh penulis (2026)

Gambar IV.15 memperlihatkan output notebook yang menjadi dasar perhitungan pada Tabel IV.8. Kolom tanggal tidak memiliki nilai kosong, sedangkan nilai kosong hanya ditemukan pada sebagian parameter polutan. Hasil ini menunjukkan bahwa permasalahan utama pada data bukan terletak pada tanggal observasi, melainkan pada kelengkapan pengukuran polutan tertentu.



Gambar IV. 16 Screenshot Output Heatmap Missing Values per Tahun dan Polutan

Sumber: Diolah oleh penulis (2026)

Heatmap pada Gambar IV.16 menunjukkan distribusi missing values berdasarkan tahun dan polutan. PM2.5 pada tahun 2020 memiliki 366 nilai kosong dari 366 hari, yang berarti seluruh observasi PM2.5 pada tahun tersebut tidak tersedia sebelum imputasi. Persentase missing PM2.5 tahun 2020 dihitung dengan membagi 366 terhadap 366 hari, kemudian dikalikan 100%, sehingga menghasilkan 100,00%. Kondisi ini perlu diperhatikan dalam interpretasi hasil, terutama ketika membaca kontribusi PM2.5 pada tahun 2020.

```
plt.show()

train_imputed, monthly_medians, global_medians = impute_pollutants(train_raw, fit=True)
test_imputed, _, _ = impute_pollutants(test_raw, monthly_medians=monthly_medians, global_medians=global_medians, fit=False)

post_missing = pd.DataFrame({
    "Subset": ["Data Latih", "Data Uji"],
    "Jumlah Missing Polutan Setelah Imputasi": [
        int(train_imputed[pollutant_cols].isna().sum().sum()),
        int(test_imputed[pollutant_cols].isna().sum().sum()),
    ],
})
save_table(post_missing, "10_missing_setelah_imputasi.csv")
display(post_missing)

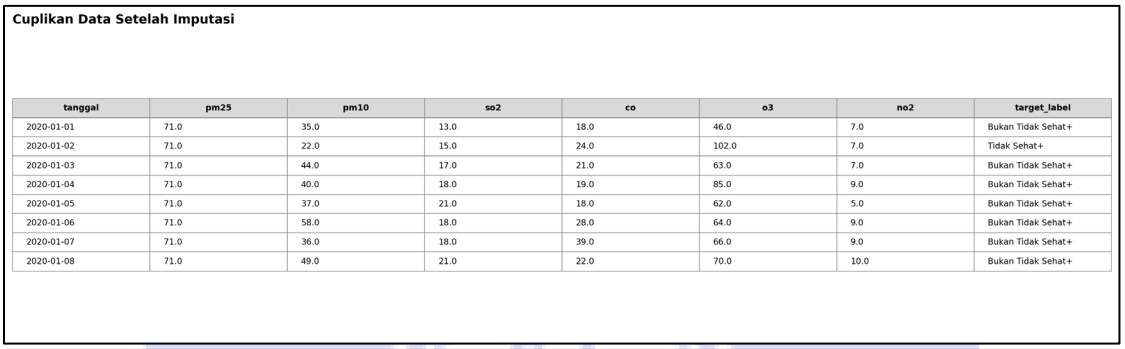
pm25_2020_missing_rate = feature_df_raw.loc[feature_df_raw["tahun"] == 2020, "pm25"].isna().mean()
display(Markdown(
    f"Proporsi nilai PM2.5 yang kosong pada tahun 2020 adalah **{pm25_2020_missing_rate:.1%}**. "
    "Interpretasi kontribusi PM2.5 pada tahun tersebut perlu dibaca dengan batasan ini."
```

Gambar IV. 17 Cuplikan Kode Python Proses Imputasi Train-Only

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.17 menunjukkan bahwa imputasi dilakukan dengan prinsip train-only. Data terlebih dahulu dibagi secara kronologis menjadi data latih dan data uji, kemudian

parameter imputasi dibentuk dari data latih. Proses pengisian nilai kosong dilakukan secara bertahap, yaitu interpolasi berbasis waktu untuk celah pendek, median bulanan data latih untuk nilai kosong yang masih tersisa, dan median global data latih sebagai langkah akhir. Prinsip ini digunakan agar informasi dari data uji tidak bocor ke proses pelatihan.



Cuplikan Data Setelah Imputasi

tanggal	pm25	pm10	so2	co	o3	no2	target_label
2020-01-01	71.0	35.0	13.0	18.0	46.0	7.0	Bukan Tidak Sehat+
2020-01-02	71.0	22.0	15.0	24.0	102.0	7.0	Tidak Sehat+
2020-01-03	71.0	44.0	17.0	21.0	63.0	7.0	Bukan Tidak Sehat+
2020-01-04	71.0	40.0	18.0	19.0	85.0	9.0	Bukan Tidak Sehat+
2020-01-05	71.0	37.0	21.0	18.0	62.0	5.0	Bukan Tidak Sehat+
2020-01-06	71.0	58.0	18.0	28.0	64.0	9.0	Bukan Tidak Sehat+
2020-01-07	71.0	36.0	18.0	39.0	66.0	9.0	Bukan Tidak Sehat+
2020-01-08	71.0	49.0	21.0	22.0	70.0	10.0	Bukan Tidak Sehat+

Gambar IV. 18 Screenshot Output Validasi Setelah Imputasi

Sumber: Diolah oleh penulis (2026)

Gambar IV.18 memperlihatkan hasil setelah proses imputasi diterapkan. Data latih dan data uji tidak lagi memiliki missing values pada fitur polutan. Hal ini memastikan bahwa seluruh model dapat dilatih dan diuji menggunakan input numerik lengkap. Output missing values dan hasil validasi imputasi dapat ditelusuri melalui tautan berikut: [https://drive.google.com/file/d/12JSryd_KwaIIvgZmT17BLWK4dtdBMLzV/view?usp=sharing].

Tabel IV. 9 Ringkasan Hasil Setelah Imputasi

Subset	Jumlah Missing Polutan Setelah Imputasi
Data latih	0
Data uji	0

Sumber: Diolah oleh penulis (2026)

Tabel IV.9 menegaskan bahwa proses imputasi berhasil menghasilkan data latih dan data uji tanpa nilai kosong pada fitur polutan. Dengan demikian, tahapan feature engineering dan pemodelan dapat dilanjutkan tanpa kehilangan baris data tambahan akibat missing values.

4.8. Feature Engineering dan Pencegahan Target Leakage

Feature engineering dilakukan untuk membentuk representasi data yang lebih informatif bagi model. Selain enam fitur polutan utama, penelitian ini membentuk fitur waktu, indikator akhir pekan, indikator pasca-pandemi, dan indikator missing values. Indikator missing values dipertahankan agar model tetap memperoleh informasi bahwa suatu nilai polutan pada awalnya kosong sebelum diimputasi.



```
[10]
✓ 0s
    leakage_check = pd.DataFrame({
        "Kolom": ["stasiun", "max", "critical", "categori", "target_tidak_sehat", "target_label"],
        "Digunakan sebagai Fitur": [col in feature_cols for col in ["stasiun", "max", "critical", "categori", "target_tidak_sehat", "target_label"]],
        "Keputusan": ["Dikecualikan"] * 6,
    })

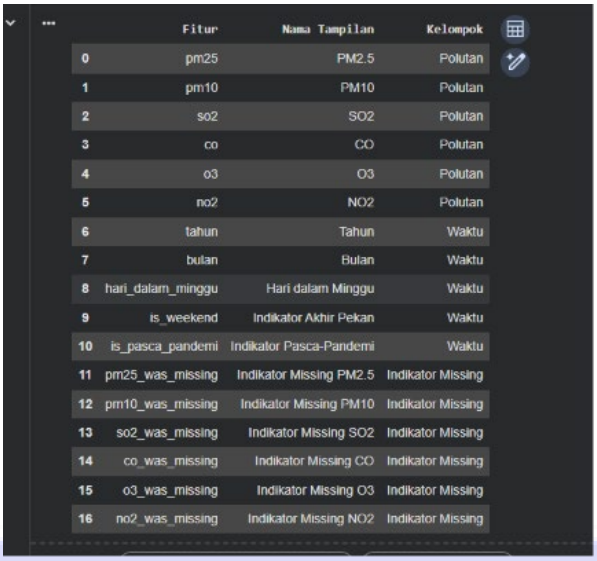
    if leakage_check["Digunakan sebagai Fitur"].any():
        raise ValueError("Terdapat kolom yang tidak boleh masuk ke fitur model.")

    save_table(leakage_check, "06_pencegahan_target_leakage.csv")
    display(leakage_check)
    print(f"Jumlah fitur sah yang digunakan: {len(feature_cols)}")
```

Gambar IV. 19 Cuplikan Kode Python Feature Engineering dan Pencegahan Target Leakage

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.19 menyusun daftar fitur final yang terdiri atas fitur polutan, fitur waktu, dan indikator missing values. Pada saat yang sama, kode tersebut juga mendefinisikan kolom yang tidak boleh digunakan sebagai input model. Kolom max dan critical dikeluarkan karena terlalu dekat dengan proses pembentukan kategori ISPU. Kolom categori, target_tidak_sehat, dan target_label dikeluarkan karena berkaitan langsung dengan target. Kolom stasiun juga dikeluarkan karena penelitian tidak diarahkan pada perbandingan antarstasiun pemantauan.



	Fitur	Nama Tampilan	Kelompok
0	pm25	PM2.5	Polutan
1	pm10	PM10	Polutan
2	so2	SO2	Polutan
3	co	CO	Polutan
4	o3	O3	Polutan
5	no2	NO2	Polutan
6	tahun	Tahun	Waktu
7	bulan	Bulan	Waktu
8	hari_dalam_minggu	Hari dalam Minggu	Waktu
9	is_weekend	Indikator Akhir Pekan	Waktu
10	is_pasca_pandemi	Indikator Pasca-Pandemi	Waktu
11	pm25_was_missing	Indikator Missing PM2.5	Indikator Missing
12	pm10_was_missing	Indikator Missing PM10	Indikator Missing
13	so2_was_missing	Indikator Missing SO2	Indikator Missing
14	co_was_missing	Indikator Missing CO	Indikator Missing
15	o3_was_missing	Indikator Missing O3	Indikator Missing
16	no2_was_missing	Indikator Missing NO2	Indikator Missing

Gambar IV. 20 Output Daftar Fitur Final

Sumber: Diolah oleh penulis (2026)

Output pada Gambar IV.20 menunjukkan bahwa fitur final model berjumlah 17. Fitur tersebut terdiri dari enam fitur polutan, lima fitur waktu atau konteks, dan enam indikator missing values. Penyusunan fitur seperti ini memungkinkan model membaca informasi polutan utama, konteks waktu, serta jejak nilai yang sebelumnya kosong tanpa memasukkan kolom yang dapat menyebabkan target leakage.

Tabel IV. 10 Ringkasan Fitur Final dan Kolom Dikecualikan

Komponen	Isi
Total fitur final	17 fitur
Fitur polutan	PM2.5, PM10, SO2, CO, O3, NO2
Fitur waktu/konteks	tahun, bulan, hari dalam minggu, indikator akhir pekan, indikator pasca-pandemi
Indikator missing	indikator missing untuk enam polutan
Kolom dikecualikan	max, critical, categori, target_tidak_sehat, target_label, stasiun

Sumber: Diolah oleh penulis (2026)

Tabel IV.10 menunjukkan bahwa total fitur final diperoleh dari 6 fitur polutan, 5 fitur waktu atau konteks, dan 6 indikator missing values. Dengan demikian, jumlah fitur final adalah 6 ditambah 5 ditambah 6, yaitu 17 fitur. File daftar fitur final dan kolom yang dikecualikan

dapat ditelusuri melalui tautan berikut: [https://drive.google.com/file/d/1KwAZ1qfPZPVIafdAB4Ly_rRqr9a9KXtT/view?usp=drive_link].

4.9. Imputasi, Scaling, dan Pembentukan Sequence LSTM/GRU

Pembagian data dilakukan secara kronologis karena data ISPU berbentuk deret waktu harian. Data tahun 2020 sampai 2023 digunakan sebagai data latih, sedangkan data tahun 2024 digunakan sebagai data uji. Pendekatan ini dipilih agar evaluasi model menyerupai kondisi penggunaan pada data masa depan, yaitu model dilatih pada periode sebelumnya dan diuji pada periode setelahnya.



```
[11]
✓ 0s
feature_df_raw = feature_df_raw.sort_values("tanggal").reset_index(drop=True)
test_start_date = pd.Timestamp("2024-01-01")

train_raw = feature_df_raw.loc[feature_df_raw["tanggal"] < test_start_date].copy()
test_raw = feature_df_raw.loc[feature_df_raw["tanggal"] >= test_start_date].copy()

y_train = train_raw["target_tidak_sehat"].copy()
y_test = test_raw["target_tidak_sehat"].copy()
train_dates = train_raw["tanggal"].copy()
test_dates = test_raw["tanggal"].copy()

split_summary = pd.DataFrame({
    "Subset": ["Data Latih", "Data Uji"],
    "Tanggal Awal": [train_dates.min().date(), test_dates.min().date()],
    "Tanggal Akhir": [train_dates.max().date(), test_dates.max().date()],
    "Jumlah Observasi": [len(train_raw), len(test_raw)],
    "Proporsi Tidak Sehat+": [round(y_train.mean(), 3), round(y_test.mean(), 3)],
})
save_table(split_summary, "07_split_data_kronologis.csv")
display(split_summary)
```

Gambar IV. 21 Cuplikan Kode Python Split Kronologis

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.21 memisahkan data berdasarkan tanggal. Observasi sebelum 1 Januari 2024 ditempatkan sebagai data latih, sedangkan observasi mulai 1 Januari 2024 ditempatkan sebagai data uji. Pembagian seperti ini menghindari pengacakan data yang dapat mengganggu urutan temporal dan berpotensi membuat informasi masa depan tercampur ke dalam proses pelatihan.



	Subset	Tanggal Awal	Tanggal Akhir	Jumlah Observasi	Proporsi Tidak Sehat+
0	Data Latih	2020-01-01	2023-12-31	1460	0.361
1	Data Uji	2024-01-01	2024-12-31	366	0.243

Gambar IV. 22 Screenshot Output Ringkasan Split Kronologis

Sumber: Diolah oleh penulis (2026)

Gambar IV.22 memperlihatkan hasil pembagian data latih dan data uji. Data latih mencakup periode 2020 sampai 2023 sebanyak 1.460 observasi, sedangkan data uji mencakup tahun 2024 sebanyak 366 observasi. Tahun 2024 digunakan sebagai data uji karena tidak digunakan dalam proses pelatihan model, sehingga sesuai untuk membaca kemampuan generalisasi model pada periode setelah data latih.

Tabel IV. 11 Ringkasan Split Data Latih dan Data Uji

Subset	Periode	Observasi	Tidak Sehat+	Perhitungan	Proporsi Tidak Sehat+
Data latih	2020-2023	1.460	527	527 / 1.460 × 100%	36,10%
Data uji	2024	366	89	89 / 366 × 100%	24,32%

Sumber: Diolah oleh penulis (2026)

Tabel IV.11 menunjukkan bahwa data latih memiliki 527 observasi kelas Tidak Sehat+ dari total 1.460 observasi. Proporsi kelas Tidak Sehat+ pada data latih dihitung dengan membagi 527 terhadap 1.460, kemudian dikalikan 100%, sehingga menghasilkan 36,10%. Data uji memiliki 89 observasi kelas Tidak Sehat+ dari total 366 observasi. Proporsi kelas Tidak Sehat+ pada data uji dihitung dengan membagi 89 terhadap 366, kemudian dikalikan 100%, sehingga menghasilkan 24,32%. Perbedaan proporsi ini menunjukkan bahwa distribusi kelas pada tahun 2024 berbeda dari periode 2020-2023, sehingga evaluasi pada data uji menjadi penting.

Cuplikan Coding Python Scaling dan Sequence

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_seq_all, y_train_seq_all, train_seq_dates = create_sequences_with_dates(
    X_train_scaled_df.values, y_train.values, train_dates.values, window_size=14
)
```

Gambar IV. 23 Cuplikan Kode Python Scaling dan Pembentukan Sequence

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.23 memperlihatkan dua proses tambahan sebelum pemodelan. Pertama, StandardScaler dihitung menggunakan data latih dan kemudian diterapkan pada data uji. Proses ini menjaga agar parameter scaling tidak mengambil informasi dari data uji. Kedua, untuk model LSTM dan GRU, data dibentuk menjadi sequence 14 hari dengan pola X[i-window:i] sebagai input dan y[i] sebagai target. Dengan pola tersebut, model deep learning memanfaatkan konteks historis 14 hari sebelumnya untuk memprediksi target pada hari tertentu.

Tabel IV. 12 Ringkasan Input Sequence LSTM dan GRU

Komponen	Perhitungan	Nilai
Window sequence	Ditentukan sebagai konteks historis 14 hari	14 hari
Sequence latih	1.460 − 14	1.446
Sequence train internal	Hasil pembagian train-validation dari 1.446 sequence	1.156
Sequence validasi internal	1.446 − 1.156	290
Target uji	Seluruh observasi tahun 2024	366

Sumber: Diolah oleh penulis (2026)

Tabel IV.12 menunjukkan bahwa jumlah sequence latih diperoleh dari jumlah data latih 1.460 dikurangi window 14 hari, sehingga menghasilkan 1.446 sequence. Sequence latih tersebut kemudian dipisahkan menjadi 1.156 sequence untuk pelatihan internal dan 290 sequence untuk validasi internal.

4.10. Training Random Forest, XGBoost, LSTM, dan GRU

Empat model dilatih untuk dibandingkan dalam penelitian ini, yaitu Random Forest, XGBoost, LSTM, dan GRU. Random Forest dan XGBoost menggunakan input tabular pada hari target, sedangkan LSTM dan GRU menggunakan sequence 14 hari sebelumnya. Perbedaan jenis input ini perlu diperhatikan dalam interpretasi hasil karena model ensemble dan model deep learning tidak menerima bentuk input yang identik.



```
[18]
✓ 0s
    model_outputs = {
        "Random Forest": {"y_true": y_test.values, "y_pred": rf_pred, "y_proba": rf_proba},
        "XGBoost": {"y_true": y_test.values, "y_pred": xgb_pred, "y_proba": xgb_proba},
        "LSTM": {"y_true": y_test_seq, "y_pred": lstm_pred, "y_proba": lstm_proba},
        "GRU": {"y_true": y_test_seq, "y_pred": gru_pred, "y_proba": gru_proba},
    }

    results_df = pd.DataFrame([
        rf_result,
        xgb_result,
        lstm_result,
        gru_result,
    ]).sort_values(["F1-Score", "ROC-AUC", "Recall", "Precision"], ascending=False).reset_index(drop=True)

    save_table(results_df.round(6), "13_evaluasi_empat_model.csv")
    display(results_df.round(4))
```

Gambar IV. 24 Cuplikan Kode Python Training Empat Model

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.24 menunjukkan skema pelatihan empat model. Random Forest menggunakan parameter class_weight="balanced" untuk menyesuaikan pembelajaran terhadap ketidakseimbangan kelas. XGBoost menggunakan parameter scale_pos_weight yang dihitung dari rasio kelas negatif terhadap kelas positif pada data latih. Pada data latih terdapat 933 observasi kelas Bukan Tidak Sehat+ dan 527 observasi kelas Tidak Sehat+, sehingga nilai scale_pos_weight yang digunakan adalah 933 / 527 = 1,77. Sementara itu, LSTM dan GRU dibangun menggunakan TensorFlow/Keras dan dilatih pada input sequence 14 hari.

Tabel IV. 13 Ringkasan Input Pemodelan

Model	Jenis Input	Data Pelatihan	Data Validasi/Uji	Output
Random Forest	Tabular hari target	Train 2020-2023 = 1.460	Test 2024 = 366	Kelas dan probabilitas Tidak Sehat+

Model	Jenis Input	Data Pelatihan	Data Validasi/Uji	Output
XGBoost	Tabular hari target	Train 2020-2023 = 1.460	Test 2024 = 366	Kelas dan probabilitas Tidak Sehat+
LSTM	Sequence 14 hari	Sequence latih = 1.446	Validasi internal = 290; test 2024 = 366	Kelas dan probabilitas Tidak Sehat+
GRU	Sequence 14 hari	Sequence latih = 1.446	Validasi internal = 290; test 2024 = 366	Kelas dan probabilitas Tidak Sehat+
Kelas negatif data latih	1.460 - 527	933	-	Dasar rasio XGBoost
Kelas positif data latih	Jumlah Tidak Sehat+ pada data latih	-	527	Dasar rasio XGBoost
scale_pos_weight XGBoost	933 / 527	933	527	1,77

Sumber: Diolah oleh penulis (2026)

Tabel IV.13 memperlihatkan perbedaan bentuk input model dan rincian parameter penting pada XGBoost. Random Forest dan XGBoost dilatih pada 1.460 observasi tabular, sedangkan LSTM dan GRU dilatih pada 1.446 sequence karena setiap input membutuhkan konteks 14 hari sebelumnya. Pada XGBoost, parameter scale_pos_weight dihitung dari rasio kelas negatif terhadap kelas positif pada data latih. Jumlah kelas negatif diperoleh dari 1.460 dikurangi 527 sehingga menghasilkan 933 observasi, sedangkan kelas positif berjumlah 527 observasi. Dengan demikian, nilai scale_pos_weight adalah 933 dibagi 527, yaitu 1,77. Nilai ini digunakan agar XGBoost lebih memperhatikan kelas Tidak Sehat+ yang jumlahnya lebih sedikit dibandingkan kelas Bukan Tidak Sehat+ pada data latih. File notebook pelatihan model dan output evaluasi awal dapat dilampirkan melalui tautan berikut:

[https://drive.google.com/file/d/1zoZmP6O0RCxMeGiFhYcaEb8zaPdGfwP4/view?usp=drive_link].

4.11. Evaluasi Model

Evaluasi model dilakukan pada data uji tahun 2024. Kelas positif dalam evaluasi ini adalah Tidak Sehat+. Metrik yang digunakan meliputi Precision, Recall, F1-Score, dan ROC-AUC. Precision menunjukkan proporsi prediksi Tidak Sehat+ yang benar, Recall menunjukkan kemampuan model menangkap seluruh kasus Tidak Sehat+, F1-Score menyeimbangkan Precision dan Recall, sedangkan ROC-AUC menunjukkan kemampuan model membedakan kelas berdasarkan probabilitas prediksi.

```
[18]
✓ 0s
    model_outputs = {
        "Random Forest": {"y_true": y_test.values, "y_pred": rf_pred, "y_proba": rf_proba},
        "XGBoost": {"y_true": y_test.values, "y_pred": xgb_pred, "y_proba": xgb_proba},
        "LSTM": {"y_true": y_test_seq, "y_pred": lstm_pred, "y_proba": lstm_proba},
        "GRU": {"y_true": y_test_seq, "y_pred": gru_pred, "y_proba": gru_proba},
    }

    results_df = pd.DataFrame([
        rf_result,
        xgb_result,
        lstm_result,
        gru_result,
    ]).sort_values(["F1-Score", "ROC-AUC", "Recall", "Precision"], ascending=False).reset_index(drop=True)

    save_table(results_df.round(6), "13_evaluasi_empat_model.csv")
    display(results_df.round(4))
```

Gambar IV. 25 Cuplikan Kode Python Evaluasi Model
Sumber: Notebook eksperimen, diolah oleh penulis (2026)

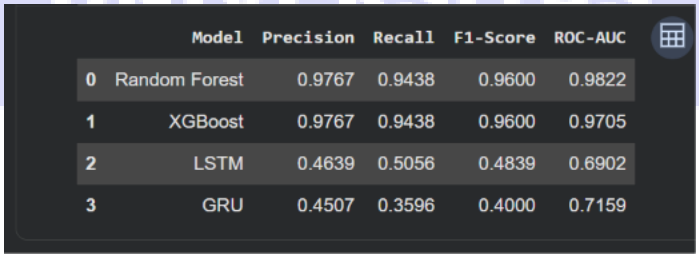
Kode pada Gambar IV.25 menyatukan target aktual, prediksi kelas, dan probabilitas kelas Tidak Sehat+ dari keempat model. Nilai tersebut digunakan untuk menghitung metrik evaluasi dan membentuk visualisasi pendukung seperti confusion matrix, grafik perbandingan performa, ROC curve, dan Precision-Recall curve. Dengan memusatkan evaluasi pada data uji 2024, hasil yang diperoleh tidak mencerminkan performa pada data latih, tetapi pada data yang tidak digunakan selama pelatihan model.

Tabel IV. 14 Hasil Evaluasi Kinerja Model

Model	Precision	Recall	F1-Score	ROC-AUC
Random Forest	84 / (84 + 2) = 0,9767	84 / (84 + 5) = 0,9438	2 × 0,9767 × 0,9438 / (0,9767 + 0,9438) = 0,9600	0,9822
XGBoost	84 / (84 + 2) = 0,9767	84 / (84 + 5) = 0,9438	2 × 0,9767 × 0,9438 / (0,9767 + 0,9438) = 0,9600	0,9705
LSTM	45 / (45 + 52) = 0,4639	45 / (45 + 44) = 0,5056	2 × 0,4639 × 0,5056 / (0,4639 + 0,5056) = 0,4839	0,6902
GRU	32 / (32 + 39) = 0,4507	32 / (32 + 57) = 0,3596	2 × 0,4507 × 0,3596 / (0,4507 + 0,3596) = 0,4000	0,7159

Sumber: Diolah oleh penulis (2026)

Tabel IV.14 menunjukkan bahwa Random Forest dan XGBoost memiliki Precision, Recall, dan F1-Score yang sama, yaitu Precision 0,9767, Recall 0,9438, dan F1-Score 0,9600. Perbedaan utama terletak pada ROC-AUC, yaitu Random Forest sebesar 0,9822 dan XGBoost sebesar 0,9705. LSTM memperoleh F1-Score 0,4839 dan ROC-AUC 0,6902, sedangkan GRU memperoleh F1-Score 0,4000 dan ROC-AUC 0,7159. Hasil ini menunjukkan bahwa model ensemble lebih stabil pada skema data tabular penelitian ini dibandingkan model sequence 14 hari yang digunakan oleh LSTM dan GRU.



	Model	Precision	Recall	F1-Score	ROC-AUC
0	Random Forest	0.9767	0.9438	0.9600	0.9822
1	XGBoost	0.9767	0.9438	0.9600	0.9705
2	LSTM	0.4639	0.5056	0.4839	0.6902
3	GRU	0.4507	0.3596	0.4000	0.7159

Gambar IV. 26 Screenshot Output Tabel Evaluasi Empat Model

Sumber: Diolah oleh penulis (2026)

Output pada Gambar IV.26 menjadi bukti langsung bahwa nilai evaluasi pada Tabel IV.14 berasal dari perhitungan notebook. Nilai Precision Random Forest dihitung dari true positive dibagi jumlah true positive dan false positive, yaitu 84 dibagi 86, sehingga

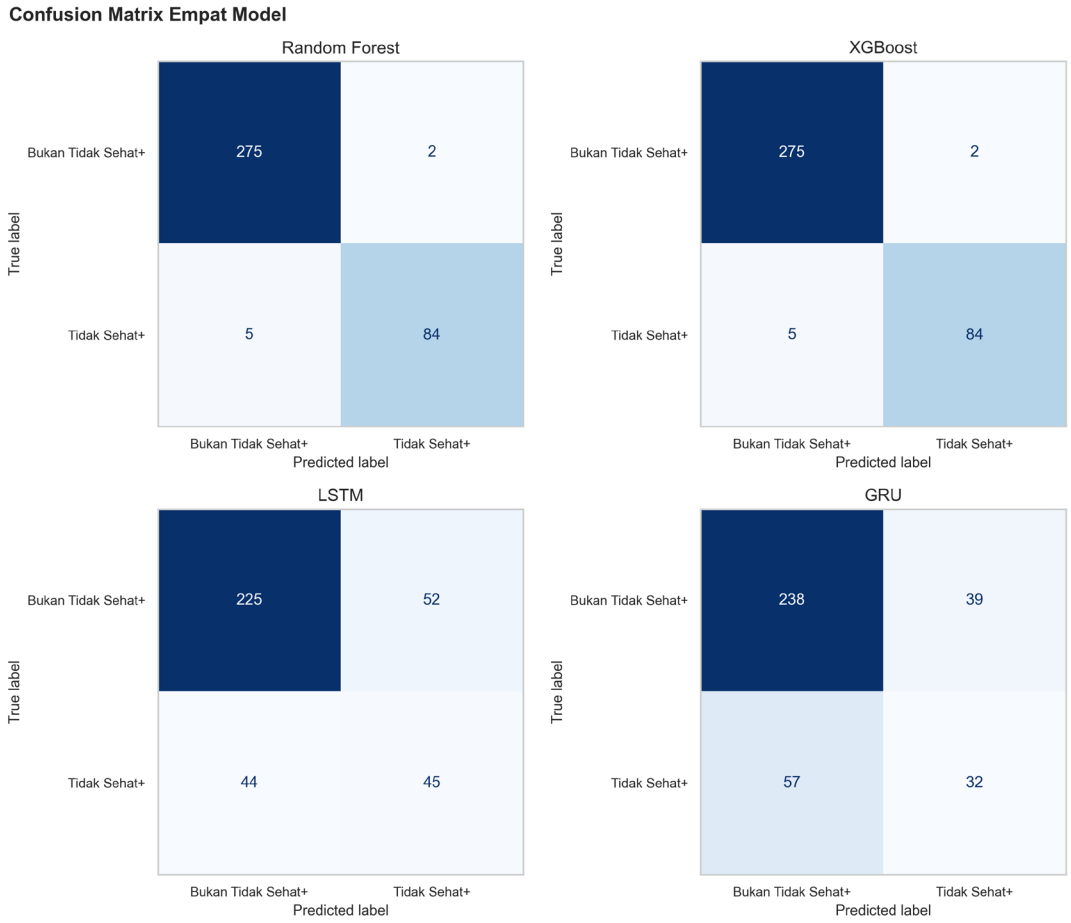
menghasilkan 0,9767. Nilai Recall dihitung dari true positive dibagi jumlah true positive dan false negative, yaitu 84 dibagi 89, sehingga menghasilkan 0,9438. F1-Score dihitung dari 2 dikalikan Precision dikalikan Recall, kemudian dibagi jumlah Precision dan Recall; 2 x 0,9767 x 0,9438 dibagi 1,9205 menghasilkan 0,9600.

Tabel IV. 15 Ringkasan Confusion Matrix Empat Model pada Data Uji 2024

Model	TN	FP	FN	TP	Precision	Recall	F1-Score
Random Forest	275	2	5	84	84 / (84 + 2) = 0,9767	84 / (84 + 5) = 0,9438	2PR / (P + R) = 0,9600
XGBoost	275	2	5	84	84 / (84 + 2) = 0,9767	84 / (84 + 5) = 0,9438	2PR / (P + R) = 0,9600
LSTM	225	52	44	45	45 / (45 + 52) = 0,4639	45 / (45 + 44) = 0,5056	2PR / (P + R) = 0,4839
GRU	238	39	57	32	32 / (32 + 39) = 0,4507	32 / (32 + 57) = 0,3596	2PR / (P + R) = 0,4000

Sumber: Diolah oleh penulis (2026)

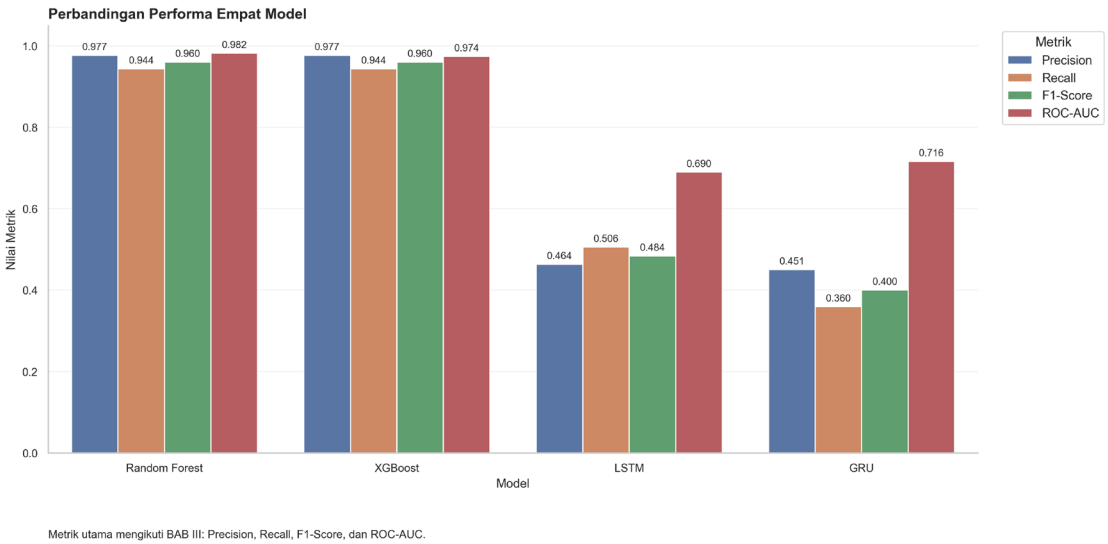
Tabel IV.15 memperlihatkan bahwa Random Forest dan XGBoost memiliki komposisi confusion matrix yang sama. Keduanya berhasil mengenali 84 hari Tidak Sehat+ dengan benar dan hanya gagal mengenali 5 hari Tidak Sehat+. Sebaliknya, LSTM dan GRU memiliki jumlah false negative yang lebih tinggi, yaitu 44 dan 57, sehingga lebih banyak hari Tidak Sehat+ yang tidak terdeteksi. Kondisi ini menjelaskan mengapa Recall dan F1-Score LSTM serta GRU lebih rendah dibandingkan model ensemble.



Gambar IV. 27 Screenshot Output Confusion Matrix Empat Model

Sumber: Diolah oleh penulis (2026)

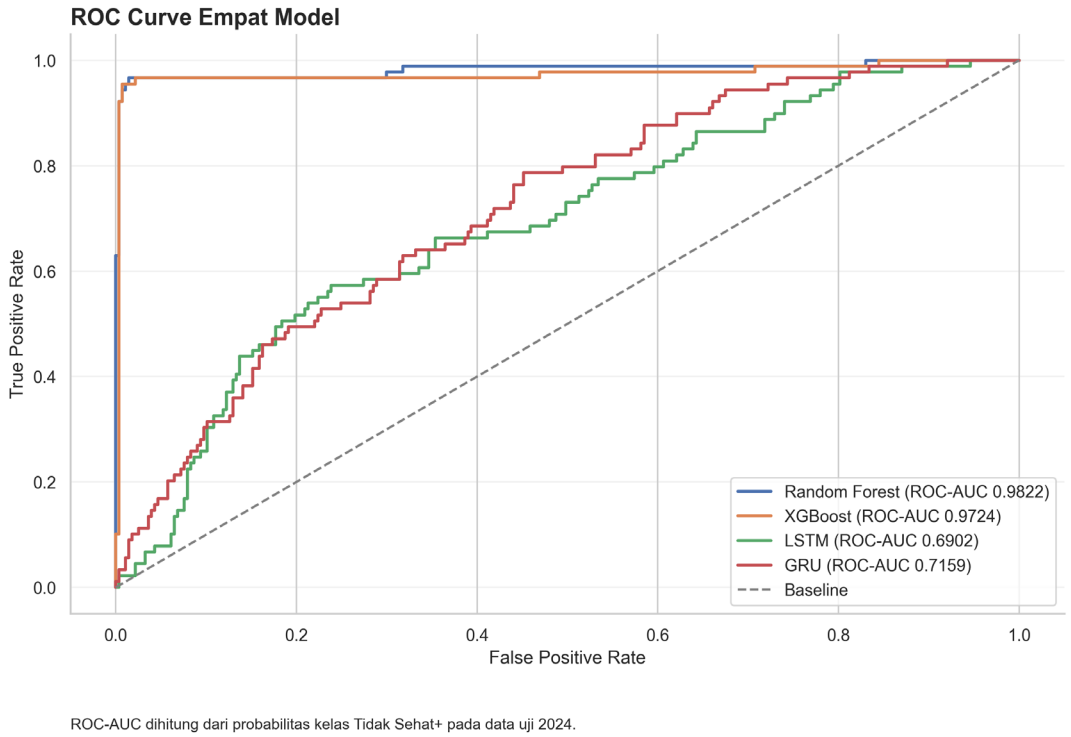
Gambar IV.27 memberikan visualisasi jumlah prediksi benar dan salah dari masing-masing model. Nilai diagonal menunjukkan prediksi benar, sedangkan nilai di luar diagonal menunjukkan kesalahan klasifikasi. Random Forest dan XGBoost menunjukkan pola kesalahan yang paling kecil pada data uji 2024, sedangkan LSTM dan GRU lebih sering mengalami kesalahan pada kelas positif Tidak Sehat+.



Gambar IV. 28 Screenshot Output Grafik Perbandingan Kinerja Model

Sumber: Diolah oleh penulis (2026)

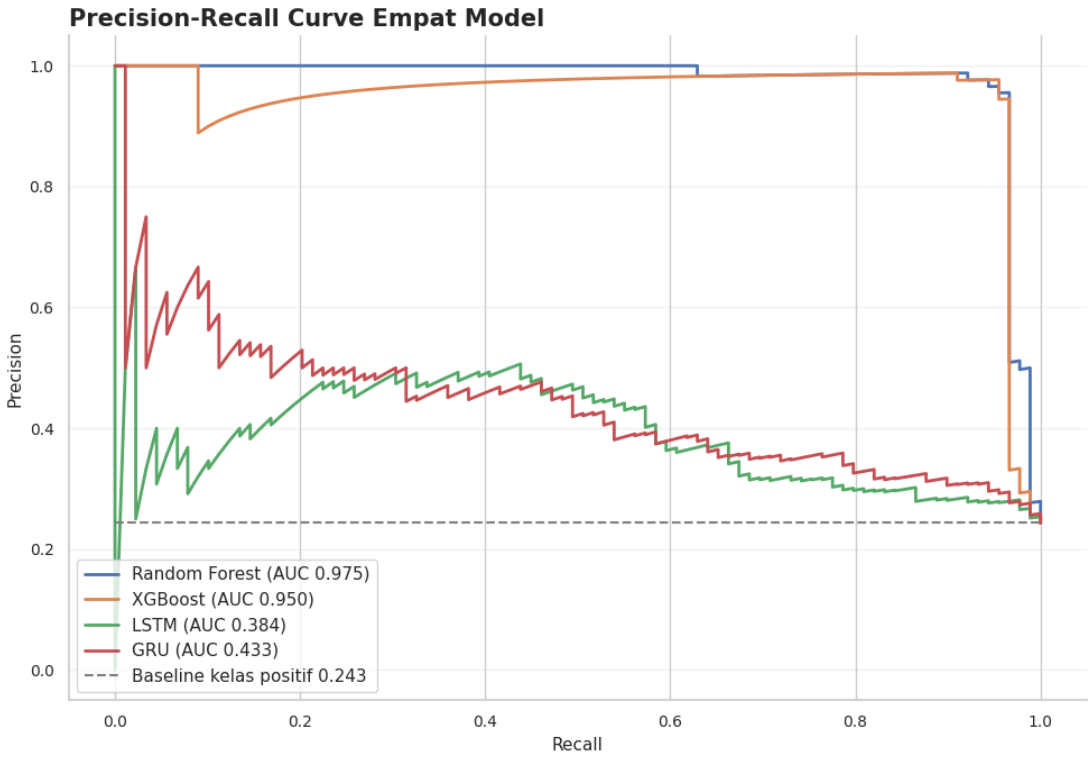
Grafik pada Gambar IV.28 memperjelas perbandingan antarmodel. Random Forest dan XGBoost memiliki batang Precision, Recall, dan F1-Score yang sangat tinggi dan hampir identik. LSTM dan GRU memiliki nilai metrik yang lebih rendah, terutama pada Recall dan F1-Score. Perbedaan ini menunjukkan bahwa pada rancangan eksperimen ini, pola tabular harian lebih mudah dimanfaatkan oleh model ensemble dibandingkan pola sequence 14 hari oleh model deep learning.



Gambar IV. 29 ROC Curve Empat Model pada Data Uji 2024

Sumber: Diolah oleh penulis (2026)

ROC curve pada Gambar IV.29 memperlihatkan kemampuan model membedakan kelas positif dan negatif berdasarkan probabilitas prediksi. Kurva Random Forest berada paling dekat dengan sudut kiri atas, yang menunjukkan kemampuan diskriminasi kelas paling baik. Hal ini sejalan dengan nilai ROC-AUC Random Forest sebesar 0,9822 pada tabel evaluasi.



Gambar IV. 30 Precision-Recall Curve Empat Model pada Data Uji 2024

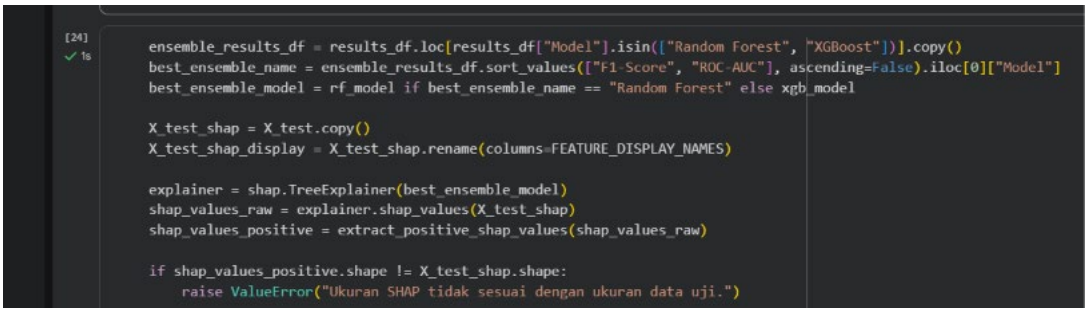
Sumber: Diolah oleh penulis (2026)

Precision-Recall curve pada Gambar IV.30 menjadi visual pendukung untuk membaca ketepatan dan kelengkapan prediksi kelas Tidak Sehat+. Kurva model ensemble menunjukkan area yang lebih baik dibandingkan LSTM dan GRU. Visual ini penting karena kelas positif Tidak Sehat+ merupakan kelas yang menjadi perhatian utama penelitian. File evaluasi model, confusion matrix, ROC curve, dan Precision-Recall curve dapat ditelusuri melalui tautan berikut: [https://drive.google.com/file/d/1hVLOx3vditU-Az9ONl9sWf8rLifsn9V2/view?usp=drive_link].

4.12. Pemilihan Model Terpilih

Setelah evaluasi model dilakukan, penelitian menetapkan model yang digunakan untuk interpretasi SHAP. Pemilihan model dilakukan pada kelompok model ensemble karena Random Forest dan XGBoost menunjukkan performa terbaik dibandingkan LSTM dan GRU.

Keduanya memiliki F1-Score yang sama, sehingga ROC-AUC digunakan sebagai pembeda utama dalam pemilihan model terpilih.



```
[24]
✓ 1s
    ensemble_results_df = results_df.loc[results_df["Model"].isin(["Random Forest", "XGBoost"])].copy()
    best_ensemble_name = ensemble_results_df.sort_values(["F1-Score", "ROC-AUC"], ascending=False).iloc[0]["Model"]
    best_ensemble_model = rf_model if best_ensemble_name == "Random Forest" else xgb_model

    X_test_shap = X_test.copy()
    X_test_shap_display = X_test_shap.rename(columns=FEATURE_DISPLAY_NAMES)

    explainer = shap.TreeExplainer(best_ensemble_model)
    shap_values_raw = explainer.shap_values(X_test_shap)
    shap_values_positive = extract_positive_shap_values(shap_values_raw)

    if shap_values_positive.shape != X_test_shap.shape:
        raise ValueError("Ukuran SHAP tidak sesuai dengan ukuran data uji.")
```

Gambar IV. 31 Cuplikan Kode Python Pemilihan Model untuk SHAP

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.31 memilih model ensemble berdasarkan F1-Score dan ROC-AUC. Karena Random Forest dan XGBoost memiliki F1-Score sama, yaitu 0,9600, model dengan ROC-AUC lebih tinggi dipilih sebagai model terpilih untuk SHAP. Dengan dasar tersebut, Random Forest dipilih karena memiliki ROC-AUC 0,9822, sedangkan XGBoost memiliki ROC-AUC 0,9705.

Tabel IV. 16 Dasar Pemilihan Model Terpilih

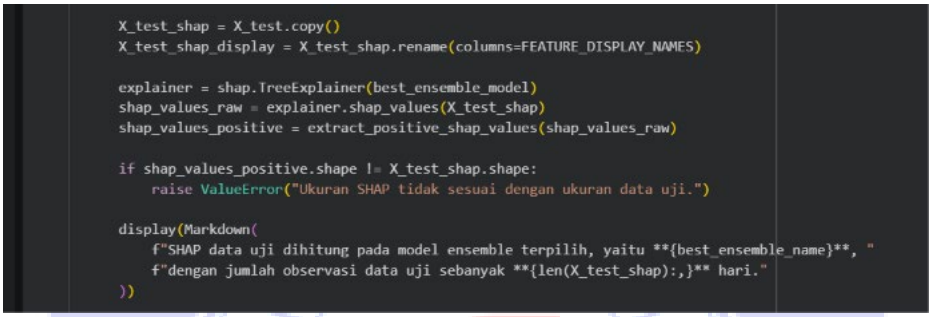
Model	F1-Score	ROC-AUC	Keputusan
Random Forest	0,9600	0,9822	Dipilih untuk interpretasi SHAP
XGBoost	0,9600	0,9724	Tidak dipilih karena ROC-AUC lebih rendah

Sumber: Diolah oleh penulis (2026)

Tabel IV.16 menunjukkan bahwa pemilihan Random Forest tidak berarti XGBoost memiliki performa buruk. Kedua model memiliki kinerja yang sangat dekat berdasarkan Precision, Recall, dan F1-Score. Namun, Random Forest dipilih karena nilai ROC-AUC yang lebih tinggi menunjukkan kemampuan diskriminasi kelas yang lebih baik pada data uji 2024. Output pemilihan model dan file pendukung SHAP dapat ditelusuri melalui tautan berikut: [https://drive.google.com/file/d/1hVLOx3voditUAz9ONl9sWf8rLifsn9V2/view?usp=drive_link].

4.13. Interpretasi SHAP Data Uji 2024

Interpretasi SHAP dilakukan untuk menjelaskan kontribusi fitur terhadap keputusan Random Forest pada data uji 2024. SHAP digunakan karena mampu memberikan ukuran kontribusi fitur pada setiap observasi dan merangkumnya menjadi kontribusi rata-rata. Interpretasi SHAP dalam penelitian ini bersifat penjelasan keputusan model, bukan bukti hubungan sebab-akibat lingkungan.



```
X_test_shap = X_test.copy()
X_test_shap_display = X_test_shap.rename(columns=FEATURE_DISPLAY_NAMES)

explainer = shap.TreeExplainer(best_ensemble_model)
shap_values_raw = explainer.shap_values(X_test_shap)
shap_values_positive = extract_positive_shap_values(shap_values_raw)

if shap_values_positive.shape != X_test_shap.shape:
    raise ValueError("Ukuran SHAP tidak sesuai dengan ukuran data uji.")

display(Markdown(
    f"SHAP data uji dihitung pada model ensemble terpilih, yaitu **{best_ensemble_name}**, "
    f"dengan jumlah observasi data uji sebanyak **{len(X_test_shap):,}** hari."
))
```

Gambar IV. 32 Cuplikan Kode Python SHAP Data Uji 2024

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.32 menghitung nilai SHAP menggunakan model Random Forest yang telah dipilih. Nilai SHAP dihitung pada data uji 2024, kemudian nilai absolut SHAP dirata-ratakan untuk setiap fitur. Hasil rata-rata absolut ini disebut mean absolute SHAP dan digunakan untuk mengetahui fitur yang paling berkontribusi terhadap keputusan model.

Tabel IV. 17 Ringkasan Mean Absolute SHAP Data Uji 2024

Peringkat	Fitur	Rumus Perhitungan	Mean Absolute SHAP
1	PM2.5	Σ|SHAP PM2.5| / 366	0,323157
2	PM10	Σ|SHAP PM10| / 366	0,083515
3	O3	Σ|SHAP O3| / 366	0,047695
4	CO	Σ|SHAP CO| / 366	0,026628
5	NO2	Σ|SHAP NO2| / 366	0,023424
6	SO2	Σ|SHAP SO2| / 366	0,013650
7	Indikator Missing PM2.5	Σ|SHAP indikator PM2.5| / 366	0,008758

Peringkat	Fitur	Rumus Perhitungan	Mean Absolute SHAP
8	Bulan	Σ|SHAP bulan| / 366	0,007862
9	Tahun	Σ|SHAP tahun| / 366	0,007825
10	Indikator Missing PM10	Σ|SHAP indikator PM10| / 366	0,005726

Sumber: Diolah oleh penulis (2026)

Tabel IV.17 menunjukkan bahwa PM2.5 memiliki kontribusi rata-rata terbesar terhadap keputusan Random Forest pada data uji 2024, yaitu 0,323157. Nilai mean absolute SHAP PM2.5 dihitung dengan menjumlahkan nilai absolut SHAP PM2.5 pada seluruh 366 observasi data uji, kemudian membaginya dengan 366. Nilai ini bukan persentase, melainkan rata-rata kontribusi absolut fitur terhadap output model. PM10 berada pada peringkat kedua dengan nilai 0,083515, diikuti O3 sebesar 0,047695, CO sebesar 0,026628, NO2 sebesar 0,023424, dan SO2 sebesar 0,013650. Hasil ini menunjukkan bahwa keputusan model Random Forest paling banyak dipengaruhi oleh informasi PM2.5 pada data uji 2024.

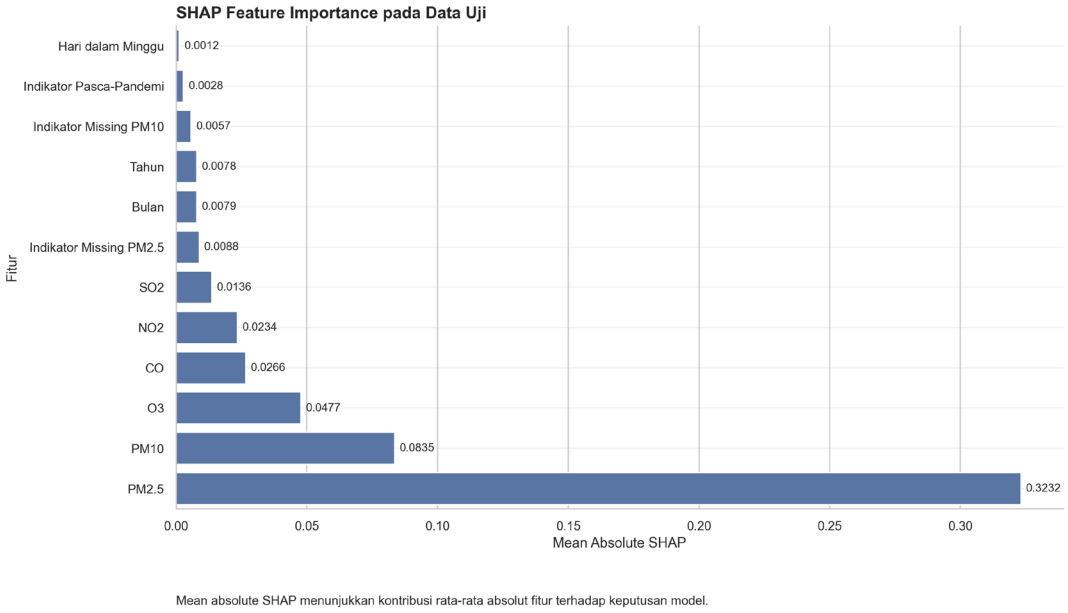


	Peringkat	Fitur	Nama Tampilan	Mean Absolute SHAP
0	1	pm25	PM2.5	0.323157
1	2	pm10	PM10	0.083515
2	3	o3	O3	0.047695
3	4	co	CO	0.026628
4	5	no2	NO2	0.023424
5	6	so2	SO2	0.013650
6	7	pm25_was_missing	Indikator Missing PM2.5	0.008758
7	8	bulan	Bulan	0.007862
8	9	tahun	Tahun	0.007825
9	10	pm10_was_missing	Indikator Missing PM10	0.005726
10	11	is_pasca_pandemi	Indikator Pasca-Pandemi	0.002825
11	12	hari_dalam_minggu	Hari dalam Minggu	0.001205

Gambar IV. 33 Screenshot Output Tabel Mean Absolute SHAP Data Uji 2024

Sumber: Diolah oleh penulis (2026)

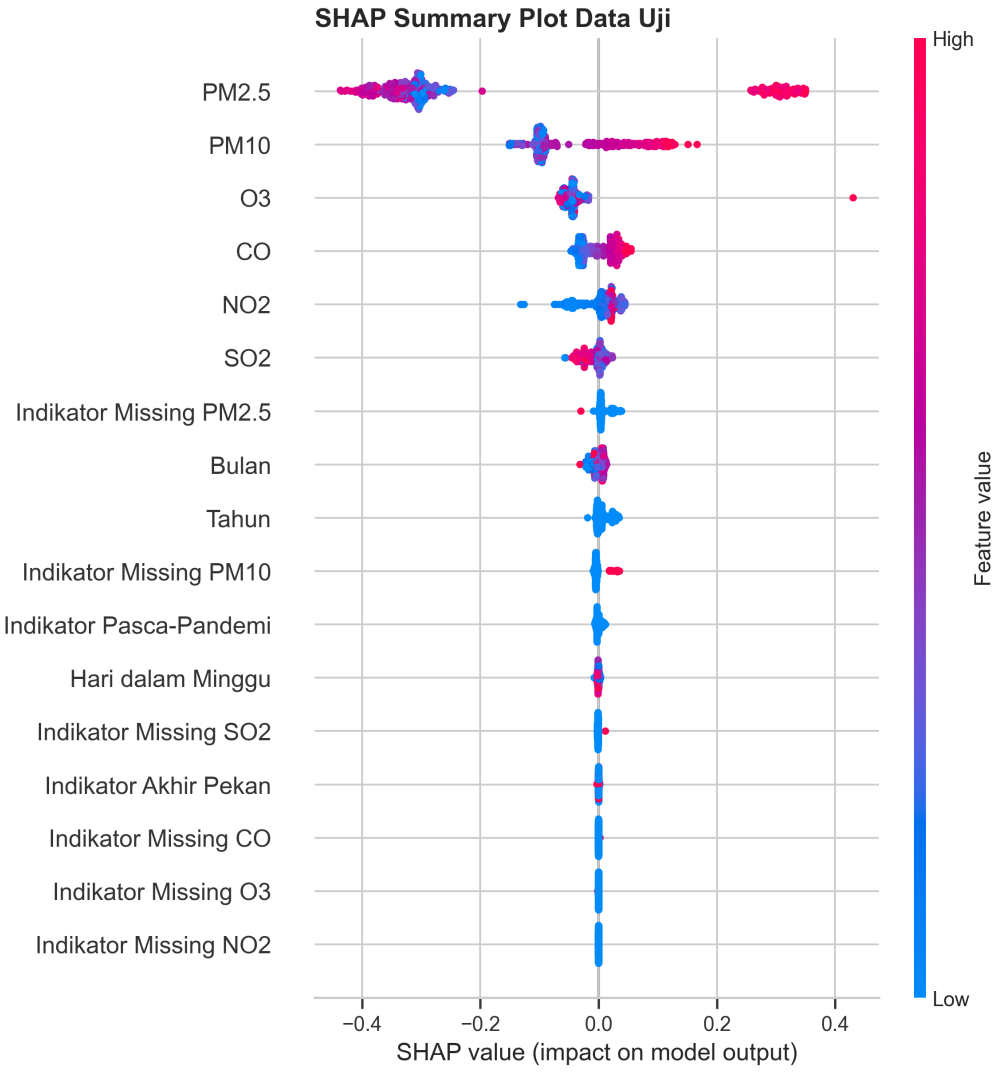
Gambar IV.33 memperlihatkan output notebook yang menjadi dasar penyusunan Tabel IV.17. Output tersebut menampilkan peringkat fitur, nama fitur, nama tampilan, dan nilai mean absolute SHAP.



Gambar IV. 34 SHAP Feature Importance pada Data Uji 2024

Sumber: Diolah oleh penulis (2026)

Gambar IV.34 menampilkan peringkat kontribusi fitur dalam bentuk grafik batang. Panjang batang menunjukkan besar mean absolute SHAP. PM2.5 tampak jauh lebih dominan dibandingkan fitur lain, sedangkan PM10 dan O3 berada di bawahnya. Visualisasi ini memperkuat pembacaan tabel bahwa partikulat halus PM2.5 menjadi fitur paling penting dalam keputusan Random Forest pada data uji 2024.



Gambar IV. 35 SHAP Summary Plot Data Uji 2024

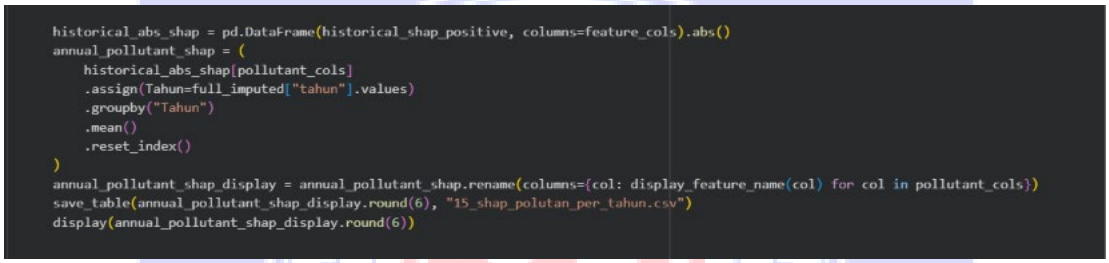
Sumber: Diolah oleh penulis (2026)

SHAP summary plot pada Gambar IV.35 memperlihatkan sebaran nilai SHAP setiap fitur pada seluruh observasi data uji. Titik yang berada ke arah kanan menunjukkan kontribusi yang mendorong prediksi ke kelas Tidak Sehat+, sedangkan titik yang berada ke arah kiri menunjukkan kontribusi yang mendorong prediksi ke kelas Bukan Tidak Sehat+. Warna titik menunjukkan nilai fitur, sehingga visual ini tidak hanya menggambarkan besar kontribusi rata-rata, tetapi juga arah pengaruh fitur terhadap keputusan model. File tabel SHAP dan gambar SHAP data uji dapat ditelusuri melalui tautan berikut: [https://drive.google.com/file/d/1DvP06zBKEjjPPzN5NsK9e7JmFEn8QSt9/view?usp=drive_link] dan

[https://drive.google.com/file/d/11kpDXnW5NX7hGmKzZgJJDV1uZ21PpQTY/view?usp=drive_link].

4.14. Interpretasi SHAP Historis 2020-2024

Interpretasi SHAP historis dilakukan untuk membaca pola kontribusi polutan dari tahun 2020 sampai 2024. Analisis ini bersifat diagnostik retrospektif karena model dilatih pada data periode 2020-2024 yang telah diimputasi untuk melihat kontribusi rata-rata polutan per tahun. Dengan demikian, hasil SHAP historis digunakan untuk memahami perubahan pola kontribusi fitur antar tahun, bukan sebagai evaluasi performa generalisasi model pada data baru.

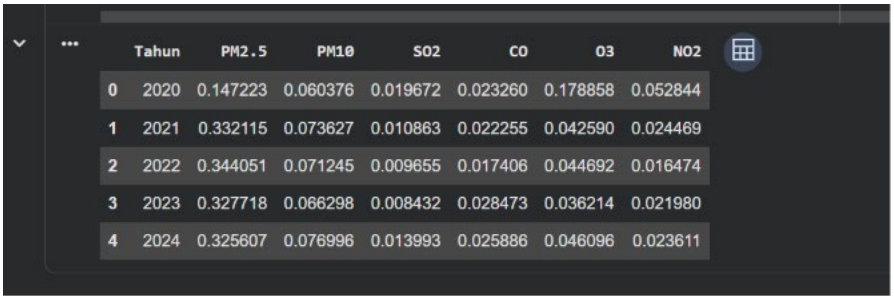


```
historical_abs_shap = pd.DataFrame(historical_shap_positive, columns=feature_cols).abs()
annual_pollutant_shap = (
    historical_abs_shap[pollutant_cols]
    .assign(Tahun=full_imputed["tahun"].values)
    .groupby("Tahun")
    .mean()
    .reset_index()
)
annual_pollutant_shap_display = annual_pollutant_shap.rename(columns={col: display_feature_name(col) for col in pollutant_cols})
save_table(annual_pollutant_shap_display.round(6), "15_shap_polutan_per_tahun.csv")
display(annual_pollutant_shap_display.round(6))
```

Gambar IV. 36 Cuplikan Kode Python SHAP Historis 2020-2024

Sumber: Notebook eksperimen, diolah oleh penulis (2026)

Kode pada Gambar IV.36 menghitung mean absolute SHAP tahunan untuk enam polutan. Nilai SHAP absolut dikelompokkan berdasarkan tahun, kemudian dirata-ratakan untuk setiap polutan. Hasil pengelompokan tersebut digunakan untuk menentukan polutan dominan dan polutan kedua pada setiap tahun.



	Tahun	PM2.5	PM10	SO2	CO	O3	NO2
0	2020	0.147223	0.060376	0.019672	0.023260	0.178858	0.052844
1	2021	0.332115	0.073627	0.010863	0.022255	0.042590	0.024469
2	2022	0.344051	0.071245	0.009655	0.017406	0.044692	0.016474
3	2023	0.327718	0.066298	0.008432	0.028473	0.036214	0.021980
4	2024	0.325607	0.076996	0.013993	0.025886	0.046096	0.023611

Ringkasan Dominasi SHAP Historis

Tahun	Polutan Dominan	Mean Absolute SHAP	Polutan Kedua	Mean Absolute SHAP Kedua
2020	O3	0.178858	PM2.5	0.147223
2021	PM2.5	0.332115	PM10	0.073627
2022	PM2.5	0.344051	PM10	0.071245
2023	PM2.5	0.327718	PM10	0.066298
2024	PM2.5	0.325607	PM10	0.076996

Gambar IV. 37 Screenshot Output Tabel Dominasi SHAP Historis

Sumber: Diolah oleh penulis (2026)

Gambar IV.37 memperlihatkan output tabel dominasi SHAP historis. Output ini menunjukkan polutan dengan nilai mean absolute SHAP tertinggi dan kedua tertinggi pada setiap tahun. Dengan menampilkan output notebook, pembahasan historis tetap dapat ditelusuri ke proses komputasi yang menghasilkan angka tersebut.

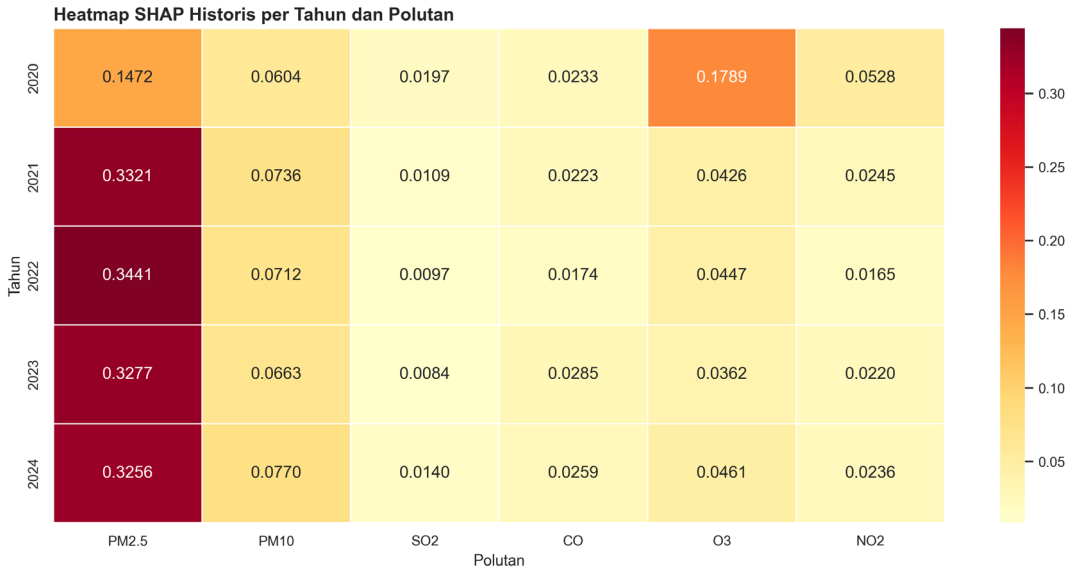
Tabel IV. 18 Ringkasan Dominasi SHAP Historis 2020-2024

Tahun	Polutan Dominan	Mean Absolute SHAP	Polutan Kedua	Mean Absolute SHAP Kedua
2020	O3	0,178858	PM2.5	0,147223
2021	PM2.5	0,332115	PM10	0,073627
2022	PM2.5	0,344051	PM10	0,071245
2023	PM2.5	0,327718	PM10	0,066298
2024	PM2.5	0,325607	PM10	0,076996

Sumber: Diolah oleh penulis (2026)

Tabel IV.18 menunjukkan bahwa O3 menjadi polutan dominan pada tahun 2020 dengan mean absolute SHAP sebesar 0,178858, sedangkan PM2.5 menjadi polutan kedua dengan nilai 0,147223. Pada tahun 2021 sampai 2024, PM2.5 menjadi polutan dominan secara konsisten.

Nilai PM2.5 pada tahun 2021 adalah 0,332115, tahun 2022 sebesar 0,344051, tahun 2023 sebesar 0,327718, dan tahun 2024 sebesar 0,325607. PM10 menjadi polutan kedua pada tahun 2021 sampai 2024. Pola ini menunjukkan bahwa PM2.5 memiliki kontribusi paling kuat dalam keputusan model pada sebagian besar periode penelitian.



Gambar IV. 38 Gambar IV.38 Heatmap SHAP Historis per Tahun dan Polutan

Sumber: Diolah oleh penulis (2026)

Heatmap pada Gambar IV.38 memperjelas perbedaan kontribusi polutan antar tahun. Warna yang lebih pekat menunjukkan nilai mean absolute SHAP yang lebih besar. PM2.5 tampak dominan pada tahun 2021 sampai 2024, sedangkan O3 menjadi fitur paling dominan pada tahun 2020. Interpretasi tahun 2020 perlu dibaca secara hati-hati karena PM2.5 pada tahun tersebut memiliki missing values penuh sebelum imputasi. Oleh karena itu, kontribusi PM2.5 tahun 2020 tidak dapat dipahami sebagai observasi asli sepenuhnya, tetapi sebagai hasil pembacaan model setelah proses imputasi. File SHAP historis dapat ditelusuri melalui tautan berikut:[https://drive.google.com/file/d/1SYgjbtdqx8U8dpPRyJSkUKNq0xsmHHN/view?usp=sharing].